

# Explicação Linha a Linha — PTV Journey Planner

Este arquivo cobre, **linha por linha**, o código de:

- `server.js`
- `models/User.js` , `models/Favorite.js` , `models/ApiKey.js`
- `routes/auth.js` , `routes/favorites.js` , `routes/apiKeys.js`
- `services/ptvSign.js` , `services/ptv.js` , `services/ptvLeg.js` , `services/journey.js` , `services/alerts.js` , `services/commonAlerts.js` , `services/places.js`
- `public/index.html` (apenas o `<script>` )
- `public/register.html` (apenas o `<script>` )

## 1. server.js

```
require('dotenv').config({ path: require('path').join(__dirname, '.env') });
```

Carrega o arquivo `.env` (que fica na mesma pasta deste arquivo, `__dirname` ) e injeta cada variável dele em `process.env` . É a primeira linha porque tudo abaixo depende dessas variáveis (chaves de API, segredo JWT etc).

```
const express = require('express');
const path = require('path');
const axios = require('axios');
const mongoose = require('mongoose');
const cors = require('cors');
```

Importa as bibliotecas principais:

- `express` — framework do servidor web
- `path` — utilitário do Node para montar caminhos de arquivos de forma segura entre sistemas operacionais
- `axios` — cliente HTTP para chamar APIs externas (Google, PTV)
- `mongoose` — ODM (Object Data Mapper) para o MongoDB
- `cors` — middleware para controlar políticas de CORS

```
const { getStop, getDepartures, getRoute } = require('./services/ptv');
const { getAlertsFromDeparture } = require('./services/alerts');
const { getJourney } = require('./services/journey');
const { getPtvLeg } = require('./services/ptvLeg');
const { getCommonAlerts } = require('./services/commonAlerts');
const { getTargetPlaces } = require('./services/places');
```

Importa as funções de cada serviço (lógica de negócio separada do `server.js` para manter o arquivo organizado). Cada uma é explicada na sua própria seção mais abaixo.

```
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./swagger');
```

`swagger-ui-express` gera a interface visual da documentação. `swaggerSpec` é o objeto JS (definido em `swagger.js`) com toda a especificação OpenAPI — rotas, parâmetros, exemplos.

```
const { v4: uuidv4, validate: uuidValidate } = require('uuid');
```

Importa duas funções da biblioteca `uuid`: `v4` gera um UUID aleatório (renomeado para `uuidv4`), `validate` checa se uma string é um UUID válido (renomeado para `uuidValidate`). Usadas nas rotas de demonstração `/uuid/generate` e `/uuid/validate`.

```
const authRoutes = require('./routes/auth');
const favoritesRoutes = require('./routes/favorites');
const apiKeysRoutes = require('./routes/apiKeys');
const apiKeyMiddleware = require('./middleware/apiKey');
```

Importa os roteadores (grupos de rotas) e o middleware de validação de API key.

```
// MongoDB
mongoose.connect(process.env.MONGODB_URI || 'mongodb://localhost:27017/ptv', { family: 4 })
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error('MongoDB connection failed:', err.message));
```

Conecta ao MongoDB. `process.env.MONGODB_URI` vem do `.env` (no nosso caso, MongoDB Atlas). Se não existir, usa um MongoDB local como fallback. `{ family: 4 }` força o uso de IPv4 — evita erros de conexão em redes onde IPv6 está mal configurado. `.then()` / `.catch()` são callbacks de Promise: o primeiro roda se conectar com sucesso, o segundo se falhar (mas o servidor continua rodando mesmo assim — não há `process.exit()`).

```
const app = express();
```

Cria a aplicação Express — o objeto principal que vamos configurar e rodar.

```
// CORS – only allow requests from known origins
const allowedOrigins = (process.env.ALLOWED_ORIGINS || 'http://localhost:3000')
  .split(',').map(o => o.trim()).filter(Boolean);
```

Lê `ALLOWED_ORIGINS` do `.env` (string separada por vírgulas, ex:

`"http://localhost:3000,https://meusite.com"`). `.split(',')` quebra em array. `.map(o => o.trim())` remove espaços em branco de cada item. `.filter(Boolean)` remove strings vazias (caso haja vírgula sobrando). Resultado: array de origens permitidas.

```
app.use(cors({
  origin: (origin, callback) => {
    // Allow non-browser requests (Postman, server-to-server) and listed origins
    if (!origin || allowedOrigins.includes(origin)) return callback(null, true);
    callback(new Error(`CORS: origin "${origin}" not allowed`));
  },
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));
```

Registra o middleware CORS globalmente (`app.use` sem path = roda em toda requisição).

- `origin` é uma função que decide se a requisição é permitida. `origin` (parâmetro) é o header `Origin` enviado pelo browser — `undefined` se for Postman/curl/server-to-server (eles não mandam esse header).
- `if (!origin || allowedOrigins.includes(origin))` — se não há header `Origin` OU se está na lista de permitidos, chama `callback(null, true)` (permite).
- Caso contrário, `callback(new Error(...))` — rejeita com erro, que cai no error handler global no fim do arquivo.
- `methods` — lista os verbos HTTP aceitos.
- `allowedHeaders` — quais headers customizados o cliente pode enviar.
- `credentials: true` — permite que o browser envie cookies/credenciais (não usamos cookies aqui, mas deixa a opção disponível).

```
// Basic security headers
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
```

```
res.setHeader('X-Frame-Options', 'DENY');
next();
});
```

Outro middleware global — função com `(req, res, next)`. Adiciona dois headers de segurança em **toda** resposta:

- `X-Content-Type-Options: nosniff` — impede o browser de tentar "adivinhar" o tipo de um arquivo diferente do `Content-Type` declarado (mitiga ataques de MIME-sniffing).
- `X-Frame-Options: DENY` — impede que a página seja carregada dentro de um `<iframe>` em outro site (mitiga clickjacking).
- `next()` — passa para o próximo middleware/rota. Sem isso, a requisição ficaria travada para sempre.

```
app.use(express.json());
```

Middleware embutido do Express: faz o parse automático do `body` de requisições com `Content-Type: application/json`, transformando o JSON recebido em objeto JS acessível via `req.body`.

```
app.use(express.static(path.join(__dirname, 'public')));
```

Serve arquivos estáticos da pasta `public/` diretamente. Por isso `index.html`, `favorites.html` etc. são acessíveis em `http://localhost:3000/index.html` (ou `/` para `index.html`) sem precisar de uma rota explícita.

```
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

Registra a documentação Swagger na rota `/api-docs`. `swaggerUi.serve` serve os arquivos estáticos da interface (CSS/JS do Swagger UI), e `swaggerUi.setup(swaggerSpec)` injeta nossa especificação OpenAPI nessa interface.

```
// Auth, Favorites & API Keys routes
app.use('/auth', authRoutes);
app.use('/favorites', favoritesRoutes);
app.use('/api-keys', apiKeysRoutes);
```

Monta os roteadores em prefixos de URL. Tudo que está definido em `routes/auth.js` (ex: `router.post('/login', ...)`) vira `/auth/login`. Mesma lógica para `/favorites` e `/api-keys`.

```
// GET /board/:stop_id?route_type=0
app.get('/board/:stop_id', apiKeyMiddleware, async (req, res) => {
```

Define a rota `GET /board/:stop_id`. `:stop_id` é um parâmetro dinâmico de rota (acessível em `req.params.stop_id`). `apiKeyMiddleware` roda **antes** do handler — se a API key for inválida, a função `async (req, res) => {...}` nem chega a executar.

```
const stopId = Number(req.params.stop_id);
const routeType = Number(req.query.route_type ?? 0);
```

`req.params.stop_id` vem como string (ex: `"1071"`); `Number(...)` converte para número. `req.query.route_type` vem da query string ( `?route_type=0` ); `?? 0` é o operador "nullish coalescing" — se `route_type` for `undefined` ou `null`, usa `0` como padrão (mas preserva `0` explícito, diferente do `||` ).

```
try {
  const [stop, departures] = await Promise.all([
    getStop(stopId, routeType),
    getDepartures(stopId, routeType)
  ]);
```

`Promise.all([...])` executa as duas chamadas ( `getStop` e `getDepartures` ) **em paralelo** em vez de sequencialmente — reduz o tempo de resposta, já que uma não depende da outra. `await` espera ambas terminarem. Destructuring `[stop, departures]` atribui o resultado de cada Promise na ordem do array.

```
const enriched = await Promise.all(departures.map(async (dep) => {
  const route = await getRoute(dep.route_id);
  const scheduledMs = new Date(dep.scheduled_departure_utc).getTime();
  const estimatedMs = new Date(dep.estimated_departure_utc).getTime();
  const delayMinutes = Math.round((estimatedMs - scheduledMs) / 60000);
```

Para cada `dep` (departure) retornado pela PTV, busca informações da rota ( `getRoute` ). `.map(async ...)` retorna um array de Promises (uma por departure) — por isso é envolvido em `Promise.all` para esperar todas. `new Date(...).getTime()` converte a string ISO de data em milissegundos desde 1970 (epoch). A diferença entre o horário estimado e o agendado, dividida por `60000` (ms em um minuto), dá o atraso em minutos. `Math.round` arredonda para inteiro.

```
return {
  route_name: route.route_name,
  route_type: route.route_type,
```

```

    platform: dep.platform_number,
    scheduled_departure: dep.scheduled_departure_utc,
    estimated_departure: dep.estimated_departure_utc,
    delay_minutes: delayMinutes,
    on_time: delayMinutes <= 0
  };
}));

```

Monta o objeto final de cada departure, já "enriquecido" com nome da rota e cálculo de atraso.

`on_time` é `true` se o atraso for zero ou negativo (chegou adiantado ou no horário).

```
const alerts = [];
```

Array vazio — reservado para alertas/disruptions desta parada. Atualmente não populado nesta rota (fica disponível para extensão futura e mantém o formato de resposta consistente).

```

res.json({
  stop: {
    stop_id: stop.stop_id,
    stop_name: stop.stop_name,
    suburb: stop.stop_suburb
  },
  departures: enriched,
  alerts
});

```

Monta a resposta JSON final: dados básicos da parada + lista de departures enriquecidos + alertas.

```

} catch (err) {
  const status = err.response?.status;
  if (status === 403 || status === 404) {
    return res.status(404).json({ error: `Stop ${req.params.stop_id} not found on PTV`
  });
  res.status(500).json({ error: err.message });
}
});

```

Tratamento de erro. `err.response?.status` — o `?.` é "optional chaining": se `err.response` for `undefined` (erro de rede, não de HTTP), não tenta acessar `.status` e retorna `undefined` em vez de lançar outro erro. Se a PTV retornou 403 ou 404 (parada não existe para esse `route_type`), respondemos com **404** e uma mensagem clara — em vez de propagar um genérico 500. Qualquer outro erro cai no 500 padrão.

```
// GET /ptv-test
app.get('/ptv-test', async (req, res) => {
  const { buildUrl } = require('./services/ptvSign');
  const url = buildUrl('/v3/route_types');
  console.log('[ptv-test] calling:', url);
  try {
    const r = await axios.get(url);
    res.json(r.data);
  } catch (err) {
    res.status(500).json({ error: err.response?.data || err.message });
  }
});
```

Rota de diagnóstico/teste — verifica se a assinatura da PTV está funcionando, chamando o endpoint mais simples possível ( `/v3/route_types` , que lista os tipos de transporte: train, tram, bus). `console.log` ajuda a debugar a URL gerada durante desenvolvimento. `err.response?.data || err.message` — prefere o corpo de erro retornado pela API (mais detalhado) e cai para `err.message` se não houver resposta HTTP.

```
// GET /uuid/generate
app.get('/uuid/generate', (req, res) => {
  res.json({ uuid: uuidv4() });
});
```

Gera e retorna um UUID v4 aleatório. Demonstra uso de uma biblioteca utilitária via API.

```
// GET /uuid/validate
app.get('/uuid/validate', (req, res) => {
  const uuid = req.headers['x-uuid'];
  if (!uuid) return res.status(400).json({ error: 'Missing x-uuid header' });
  const valid = uuidValidate(uuid);
  res.json({ uuid, valid });
});
```

Lê o header customizado `x-uuid` (headers HTTP são sempre lowercase em `req.headers` ). Se ausente, retorna **400 Bad Request**. Caso contrário, valida o formato com `uuidValidate` e retorna `{ uuid, valid: true/false }` .

```
// GET /config
app.get('/config', (req, res) => {
```

```
res.json({ mapsApiKey: process.env.GOOGLE_MAPS_API_KEY });
});
```

Endpoint que o **frontend** chama para obter a chave do Google Maps em tempo de execução. Isso evita "hardcodar" a chave dentro do HTML/JS estático — ela só sai do `.env` quando o frontend pede.

```
// GET /reverse-geocode?lat=...&lng=...
app.get('/reverse-geocode', async (req, res) => {
  const { lat, lng } = req.query;
  if (!lat || !lng) return res.status(400).json({ error: 'lat and lng are required' });
  try {
    const r = await axios.get('https://maps.googleapis.com/maps/api/geocode/json', {
      params: { latlng: `${lat},${lng}`, key: process.env.GOOGLE_MAPS_API_KEY, language:
    });
    const result = r.data.results[0];
    if (!result) return res.status(404).json({ error: 'No address found' });
    res.json({ address: result.formatted_address });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Recebe latitude/longitude (do GPS do navegador). Valida que ambos existem (**400** se faltarem). Chama o Geocoding API do Google passando `latlng` no formato `"lat,lng"`. `r.data.results[0]` é o resultado mais relevante — se não houver nenhum, **404**. Retorna apenas o endereço formatado (string legível).

```
// GET /autocomplete?input=...
app.get('/autocomplete', async (req, res) => {
  const { input } = req.query;
  if (!input) return res.json({ suggestions: [] });
```

Se `input` (texto digitado pelo usuário) estiver vazio, retorna lista vazia imediatamente — evita chamada desnecessária ao Google.

```
try {
  const r = await axios.get(
    'https://maps.googleapis.com/maps/api/place/autocomplete/json',
    {
      params: {
        input,
```



```

    key: process.env.GOOGLE_MAPS_API_KEY,
    components: 'country:au',
    location: '-37.8136,144.9631',
    radius: 50000,
    language: 'en'
  }
}
);

```

Chama o Places Autocomplete. `components: 'country:au'` restringe a resultados na Austrália. `location` + `radius: 50000` (50km) cria um viés de busca ao redor do centro de Melbourne ( `-37.8136, 144.9631` ), priorizando resultados próximos sem excluir totalmente os distantes.

```

const suggestions = (r.data.predictions || []).map(p => ({
  text: p.description,
  placeId: p.place_id
})));
res.json({ suggestions });
} catch (err) {
  const detail = err.response?.data || err.message;
  console.error('[autocomplete]', JSON.stringify(detail));
  res.status(500).json({ error: detail });
}
});

```

`r.data.predictions || []` — se o Google não retornar `predictions` (campo ausente), usa array vazio para evitar erro no `.map`. Transforma cada predição em um objeto simplificado `{ text, placeId }` — só o que o frontend precisa. No erro, loga o detalhe completo no console do servidor (debug) e retorna 500 ao cliente.

```

// GET /target-places?destination=...&categories=coffee,hotel,restaurant
app.get('/target-places', async (req, res) => {
  const { destination, categories } = req.query;
  if (!destination) return res.status(400).json({ error: 'destination is required' });

  try {
    const result = await getTargetPlaces(destination, categories, process.env.GOOGLE_MAPS_API_KEY);
    if (!result) return res.status(404).json({ error: 'Destination could not be located' });
    res.json(result);
  } catch (err) {
    const detail = err.response?.data || err.message;
    console.error('[target-places]', JSON.stringify(detail));
    res.status(500).json({ error: detail });
  }
});

```

```

    }
  });

```

Valida `destination` obrigatório (**400**). Delega toda a lógica para `getTargetPlaces()` (em `services/places.js`). Se a função retornar `null` (não conseguiu geocodificar o destino), responde **404**. Caso contrário, repassa o resultado.

```

// GET /journey?origin=...&destination=...
app.get('/journey', async (req, res) => {
  const { origin, destination } = req.query;
  if (!origin || !destination) {
    return res.status(400).json({ error: 'origin and destination are required' });
  }

```

Rota principal do app. Valida que ambos os parâmetros existem.

```

try {
  const journey = await getJourney(origin, destination);
  if (!journey) return res.status(404).json({ error: 'No route found' });

```

Chama o Google Routes API (via `services/journey.js`). Se não houver rota possível, **404**.

```

const ROUTE_TYPE = { HEAVY_RAIL: 0, COMMUTER_TRAIN: 0, RAIL: 0, TRAM: 1, BUS: 2 };

```

Mapa de conversão: o Google retorna o modo de transporte como string ( `"TRAM"`, `"BUS"` etc), mas a PTV usa números ( `0` = trem, `1` = tram, `2` = ônibus). Esse objeto faz a tradução.

```

const enrichedLegs = await Promise.all(journey.legs.map(async (leg) => {
  const routeType = ROUTE_TYPE[leg.mode] ?? 0;
  const ptv = await getPtvLeg(leg.departure_stop, routeType, leg.line_short, leg.dep
    console.error('[ptv error]', err.response?.data || err.message);
    return { found: false };
  });
  const disruptions = getAlertsFromDeparture(leg.departure_time, ptv?.departures?.[0]
  return { ...leg, ptv, disruptions };
}));

```

Para cada trecho ( `leg` ) da rota do Google:

- `routeType = ROUTE_TYPE[leg.mode] ?? 0` — converte o modo para o número da PTV; se não estiver no mapa, assume `0` (trem).

- `getPtvLeg(...)` busca as próximas partidas PTV para a parada de origem deste trecho. `.catch(err => {...})` — se a chamada PTV falhar (parada não encontrada, erro de rede), **não quebra a requisição inteira**: loga o erro e retorna `{ found: false }`, permitindo que o resto da viagem ainda seja exibido.
- `getAlertsFromDeparture(...)` compara o horário em tempo real do Google (`leg.departure_time`) com o horário agendado da PTV (`ptv?.departures?.[0]?.scheduled_departure` — o `?.` evita erro se `ptv.departures` for `undefined` ou vazio) para detectar atraso.
- `{ ...leg, ptv, disruptions }` — spread operator: copia todas as propriedades de `leg` e adiciona `ptv` e `disruptions` ao objeto.

```
res.json({ ...journey, legs: enrichedLegs });
} catch (err) {
  const detail = err.response?.data || err.message;
  console.error('[journey]', JSON.stringify(detail));
  res.status(500).json({ error: detail });
}
});
```

Retorna o objeto `journey` original, mas substituindo `legs` pela versão enriquecida (`{ ...journey, legs: enrichedLegs }` sobrescreve a propriedade `legs`).

```
// GET /common-alerts?route_type=0
app.get('/common-alerts', apiKeyMiddleware, async (req, res) => {
  const routeType = Number(req.query.route_type ?? 0);
  try {
    const { alerts } = await getCommonAlerts(routeType);
    res.json({
      generated_at: new Date().toISOString(),
      route_type: routeType,
      alert_count: alerts.length,
      alerts
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Rota de developer (exige API key). Busca disruptions gerais da PTV para o tipo de transporte informado. `generated_at: new Date().toISOString()` — timestamp de quando a resposta foi gerada, útil para quem consome a API saber a "idade" dos dados. `alert_count` é calculado a partir do tamanho do array — conveniência para quem consome sem precisar contar.

```
// GET /ptv-leg?origin=...&destination=...&route_type=0
app.get('/ptv-leg', apiKeyMiddleware, async (req, res) => {
  const { origin, destination, route_type } = req.query;
  if (!origin || !destination) {
    return res.status(400).json({ error: 'origin and destination are required' });
  }
  try {
    const result = await getPtvLeg(origin, Number(route_type ?? 0));
    if (result.error) return res.status(404).json({ error: result.error });
    res.json(result);
  } catch (err) {
    const detail = err.response?.data || err.message;
    res.status(500).json({ error: detail });
  }
});
```

Outra rota de developer. Permite consultar diretamente as partidas PTV de uma parada pelo nome (sem precisar passar por `/journey`). `result.error` é setado por `getPtvLeg` quando a parada não é encontrada ( `{ found: false, error: '...' }` ) — nesse caso, **404**.

Nota: `destination` é exigido na validação mas não é usado dentro de `getPtvLeg` nesta rota — mantido para consistência da assinatura da API e possíveis extensões futuras (ex.: calcular o trecho entre origem e destino).

```
// 404
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});
```

Middleware "catch-all" — só é alcançado se **nenhuma** rota acima deu match. Captura qualquer URL inexistente e retorna **404** em JSON (em vez do HTML padrão de erro do Express).

```
// Global error handler
app.use((err, req, res, next) => {
  console.error('[error]', err.message);
  res.status(err.status || 500).json({ error: err.message || 'Internal server error' });
});
```

Middleware de erro do Express — reconhecido por ter **4 parâmetros** ( `err, req, res, next` ). É chamado quando algum middleware/rota chama `next(err)` ou lança uma exceção não tratada

(ex: o erro de CORS criado com `new Error(...)`). `err.status || 500` — usa o status definido no erro, ou 500 como padrão.

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Departure Board running at http://localhost:${PORT}`);
  console.log(`Google API Key: ${process.env.GOOGLE_MAPS_API_KEY ? '✓ loaded' : 'X MISSING'}`);
  console.log(`JWT Secret: ${process.env.JWT_SECRET ? '✓ loaded' : 'X MISSING'}`);
});
```

`PORT` vem do `.env` ou usa `3000` como padrão (Render/Heroku, por exemplo, definem `PORT` automaticamente). `app.listen(PORT, callback)` inicia o servidor HTTP — o callback roda assim que o servidor está pronto, exibindo logs úteis para confirmar (sem expor os valores reais) que as variáveis de ambiente sensíveis foram carregadas com sucesso.

## 2. models/User.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
```

`mongoose` para definir o schema/model. `bcryptjs` é a biblioteca de hashing de senhas (versão em JS puro do `bcrypt`, sem dependências nativas — mais fácil de instalar no Windows).

```
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 6 },
}, { timestamps: true });
```

Define o formato de um documento `User` no MongoDB:

- `email` — string obrigatória, **única** (o MongoDB cria um índice que rejeita duplicatas), `lowercase: true` converte automaticamente para minúsculas antes de salvar (evita ter `Joao@x.com` e `joao@x.com` como contas diferentes), `trim: true` remove espaços nas pontas.
- `password` — string obrigatória, mínimo 6 caracteres (validado pelo Mongoose antes de salvar).
- `{ timestamps: true }` — opção do schema que faz o Mongoose adicionar e manter automaticamente os campos `createdAt` e `updatedAt`.

```

userSchema.pre('save', async function () {
  if (this.isModified('password')) {
    this.password = await bcrypt.hash(this.password, 10);
  }
});

```

Hook (gancho) que roda **antes** de qualquer `.save()` no banco. `this` é o documento sendo salvo. `this.isModified('password')` retorna `true` apenas se o campo `password` foi alterado nesta operação — importante porque, se você atualizar só o email de um usuário existente, esse código não vai re-hashear (e quebrar) a senha já hasheada. `bcrypt.hash(senha, 10)` gera o hash com **10 "salt rounds"** (custo computacional — quanto maior, mais lento e mais seguro contra força bruta). O resultado substitui a senha em texto puro antes de ir para o banco.

```

userSchema.methods.comparePassword = function (plain) {
  return bcrypt.compare(plain, this.password);
};

```

Adiciona um **método de instância** ao schema — qualquer documento `User` carregado do banco terá essa função disponível ( `user.comparePassword(...)` ). `bcrypt.compare(senhaDigitada, hashSalvo)` retorna uma Promise — o bcrypt extrai o salt do hash salvo, aplica o mesmo algoritmo na senha digitada e compara os resultados. Nunca "descripta" o hash — comparação é unidirecional.

```

module.exports = mongoose.model('User', userSchema);

```

Registra o schema como um **model** chamado `'User'` (o Mongoose cria/usa a coleção `users` no MongoDB — pluraliza e deixa em minúsculas automaticamente) e exporta a classe resultante, que é usada em `routes/auth.js` para criar/buscar usuários.

### 3. models/Favorite.js

```

const mongoose = require('mongoose');

const favoriteSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  name: { type: String, required: true, trim: true },
  origin: { type: String, required: true, trim: true },
  destination: { type: String, required: true, trim: true },
});

```

- `userId` — tipo `ObjectId` (o tipo de ID padrão do MongoDB, 12 bytes). `ref: 'User'` diz ao Mongoose que esse campo referencia um documento da coleção `User` (permite usar `.populate()` no futuro para "juntar" os dados, embora não seja usado atualmente). `index: true` cria um índice no banco — torna `Favorite.find({ userId })` muito mais rápido, já que essa é a consulta mais comum (listar favoritos de um usuário).
- `name`, `origin`, `destination` — strings obrigatórias com `trim` (remove espaços extras digitados pelo usuário).

```
places: [{
  place_id: { type: String, trim: true },
  name: { type: String, trim: true },
  address: { type: String, trim: true },
  category: { type: String, trim: true },
  location: {
    lat: Number,
    lng: Number
  },
  distance_meters: Number,
  walking_minutes: Number,
  rating: Number,
  user_ratings_total: Number,
  open_now: Boolean
}]
}, { timestamps: true });
```

`places` é um **array de subdocumentos** — cada rota favorita pode ter zero ou mais "pontos de interesse" associados (cafés, restaurantes etc., selecionados pelo usuário no app). Nenhum desses campos é `required` — um lugar pode ter dados parciais (ex: API do Google não retornou rating). `location: { lat, lng }` é um objeto aninhado simples (não outro `ObjectId` /referência).

```
module.exports = mongoose.model('Favorite', favoriteSchema);
```

Cria o model `Favorite` (coleção `favorites`) e exporta — usado em `routes/favorites.js`.

## 4. models/ApiKey.js

```
const mongoose = require('mongoose');

const apiKeySchema = new mongoose.Schema({
```

```
key: { type: String, required: true, unique: true },
name: { type: String, required: true, trim: true },
userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
active: { type: Boolean, default: true }
}, { timestamps: true });
```

- **key** — a string da chave gerada (ex: `ptv_a3f8...`), **única** no banco — duas chaves nunca podem colidir.
- **name** — apelido dado pelo usuário para identificar a chave (ex: "Minha integração Postman").
- **userId** — referência ao dono da chave (mesmo padrão do **Favorite**).
- **active** — booleano com **default: true**. Quando o usuário "revoga" a chave, em vez de deletar o documento, poderia setar isso para **false** (na implementação atual, **DELETE /api-keys/:id** remove o documento — mas o campo **active** ainda é checado pelo middleware, cobrindo o caso de chaves desativadas sem exclusão).

```
module.exports = mongoose.model('ApiKey', apiKeySchema);
```

Cria o model **ApiKey** (coleção **apikeys**) — usado em **routes/apiKeys.js** e **middleware/apiKey.js**.

## 5. routes/auth.js

```
const router = require('express').Router();
const jwt = require('jsonwebtoken');
const User = require('../models/User');
```

**Router()** cria um "mini app" Express que agrupa rotas relacionadas — depois é montado no **server.js** com **app.use('/auth', authRoutes)**. **jsonwebtoken** é a biblioteca para criar/verificar tokens JWT. **User** é o model do MongoDB.

```
// POST /auth/register
router.post('/register', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ error: 'Email and password are required' });
  }
  if (password.length < 6) {
```



```
return res.status(400).json({ error: 'Password must be at least 6 characters' });
}
```

Lê `email` e `password` do corpo JSON da requisição (graças ao `express.json()` no `server.js`). Duas validações antes de tocar no banco: campos presentes, e senha com pelo menos 6 caracteres (mesma regra do schema, mas validar aqui dá uma mensagem de erro mais amigável e evita uma viagem desnecessária ao banco).

```
try {
  const exists = await User.findOne({ email });
  if (exists) return res.status(409).json({ error: 'Email already registered' });
}
```

`User.findOne({ email })` busca um usuário com esse email. Se já existir, retorna **409 Conflict** — código HTTP correto para "o recurso já existe e conflita com o que você está tentando criar".

```
const user = await User.create({ email, password });
```

Cria o documento no MongoDB. Nesse momento, o hook `pre('save')` do `User.js` roda automaticamente e transforma `password` em hash antes de gravar.

```
const token = jwt.sign(
  { id: user._id, email: user.email },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);
res.status(201).json({ token, user: { id: user._id, email: user.email } });
```

`jwt.sign(payload, secret, options)` gera o token:

- **payload** — dados que ficam codificados (não criptografados!) dentro do token: `id` e `email` do usuário.
- **secret** — `JWT_SECRET` do `.env`, usado para assinar digitalmente o token (garante que ninguém pode forjar ou alterar o conteúdo sem ter esse segredo).
- `expiresIn: '7d'` — o token deixa de ser válido automaticamente após 7 dias.

`res.status(201)` — **201 Created**, código correto para "novo recurso criado com sucesso". Retorna o token (para o frontend já considerar o usuário logado) e os dados públicos do usuário (nunca a senha/hash).

```
} catch (err) {
  res.status(500).json({ error: err.message });
}
```

```
}  
});
```

Captura qualquer erro inesperado (ex: falha de conexão com o MongoDB) e retorna **500**.

```
// POST /auth/login  
router.post('/login', async (req, res) => {  
  const { email, password } = req.body;  
  if (!email || !password) {  
    return res.status(400).json({ error: 'Email and password are required' });  
  }  
  try {  
    const user = await User.findOne({ email });  
    if (!user) return res.status(401).json({ error: 'Invalid credentials' });  
  
    const valid = await user.comparePassword(password);  
    if (!valid) return res.status(401).json({ error: 'Invalid credentials' });  
  }  
});
```

Busca o usuário pelo email. Se não existir, **401 Unauthorized**. Se existir,

`user.comparePassword(password)` (definido em `models/User.js`) compara a senha digitada com o hash salvo. Se não bater, também **401**.

**Detalhe de segurança:** a mensagem de erro é a mesma ( `'Invalid credentials'` ) tanto para "email não existe" quanto para "senha errada". Isso evita que um atacante descubra quais emails estão cadastrados testando um por um (user enumeration).

```
const token = jwt.sign(  
  { id: user._id, email: user.email },  
  process.env.JWT_SECRET,  
  { expiresIn: '7d' }  
);  
res.json({ token, user: { id: user._id, email: user.email } });  
} catch (err) {  
  res.status(500).json({ error: err.message });  
}  
});  
  
module.exports = router;
```

Mesma geração de token do registro. `res.json(...)` sem `.status(...)` explícito — Express usa **200 OK** por padrão. `module.exports = router` exporta o roteador para ser montado no `server.js`.

## 6. routes/favorites.js

```
const router = require('express').Router();
const Favorite = require('../models/Favorite');
const auth = require('../middleware/auth');
```

Importa o model e o middleware de autenticação JWT ( `middleware/auth.js` ) — será aplicado em **todas** as rotas deste arquivo.

```
function normalizePlaces(places = []) {
  if (!Array.isArray(places)) return [];
  return places.filter(p => p && (p.place_id || p.name)).map(place => ({
```

Função auxiliar (não é uma rota). `places = []` — valor padrão caso `places` seja `undefined`. `Array.isArray(places)` — se o cliente mandar algo que não é array (ex: um objeto ou string), retorna array vazio em vez de quebrar. `.filter(p => p && (p.place_id || p.name))` — descarta entradas `null` / `undefined` ou que não tenham nem `place_id` nem `name` (dados inúteis).

```
    place_id: place.place_id ? String(place.place_id).trim() : undefined,
    name: place.name ? String(place.name).trim() : undefined,
    address: place.address ? String(place.address).trim() : undefined,
    category: place.category ? String(place.category).trim() : undefined,
    location: {
      lat: place.location?.lat != null ? Number(place.location.lat) : undefined,
      lng: place.location?.lng != null ? Number(place.location.lng) : undefined
    },
    distance_meters: place.distance_meters != null ? Number(place.distance_meters) : undefined,
    walking_minutes: place.walking_minutes != null ? Number(place.walking_minutes) : undefined,
    rating: place.rating != null ? Number(place.rating) : undefined,
    user_ratings_total: place.user_ratings_total != null ? Number(place.user_ratings_total) : undefined,
    open_now: place.open_now === true
  }));
}
```

Para cada campo, garante o **tipo correto** antes de salvar no MongoDB:

- Strings: `String(valor).trim()` se existir, senão `undefined` (campo fica de fora do documento).

- `location.lat / lng : place.location?.lat != null` — `?.` evita erro se `location` for `undefined`; `!= null` cobre tanto `null` quanto `undefined` (mas não `0`, que é um valor válido de coordenada). Converte para `Number`.
- Campos numéricos ( `distance_meters`, `rating` etc.) — mesma lógica: converte para `Number` só se não for `null / undefined`.
- `open_now: place.open_now === true` — força booleano estrito; qualquer valor que não seja literalmente `true` (incluindo `"true"` string, `1`, `undefined`) vira `false`.

Esse "saneamento" evita que dados malformados vindos do frontend (ou de uma chamada de API mal-intencionada) corrompam o schema.

```
// GET /favorites – list all favorites for the logged-in user
router.get('/', auth, async (req, res) => {
  try {
    const favorites = await Favorite.find({ userId: req.user.id }).sort('-createdAt');
    res.json(favorites);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

`auth` é o segundo argumento — o **middleware** roda antes do handler. Se o JWT for inválido/ausente, a requisição nunca chega aqui (o middleware já responde 401). `req.user.id` vem do payload do token (decodificado pelo `jwt.verify` dentro do middleware).

`Favorite.find({ userId: req.user.id })` — só retorna favoritos **deste usuário** (isolamento de dados entre contas). `.sort('-createdAt')` — ordena do mais recente para o mais antigo ( `-` = decrescente).

```
// POST /favorites – create a new favorite
router.post('/', auth, async (req, res) => {
  const { name, origin, destination, places } = req.body;
  if (!name || !origin || !destination) {
    return res.status(400).json({ error: 'name, origin and destination are required' });
  }
  try {
    const normalizedPlaces = normalizePlaces(places);
    const fav = await Favorite.create({ userId: req.user.id, name, origin, destination,
    res.status(201).json(fav);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Valida os 3 campos obrigatórios (**400** se faltar algum). `places` é opcional — passa por `normalizePlaces` (que lida com `undefined` retornando `[]`). `userId: req.user.id` é setado automaticamente a partir do token — o cliente **não pode** escolher para qual usuário a rota é salva. **201 Created** com o documento criado (incluindo o `_id` gerado pelo MongoDB).

```
// PUT /favorites/:id - full update (all fields required)
router.put('/:id', auth, async (req, res) => {
  const { name, origin, destination, places } = req.body;
  if (!name || !origin || !destination) {
    return res.status(400).json({ error: 'name, origin and destination are required' });
  }
  try {
    const fav = await Favorite.findOne({ _id: req.params.id, userId: req.user.id });
    if (!fav) return res.status(404).json({ error: 'Favorite not found' });
    fav.name = name;
    fav.origin = origin;
    fav.destination = destination;
    fav.places = normalizePlaces(places);
    await fav.save();
    res.json(fav);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

PUT exige **todos** os campos principais (semântica de "substituição completa" do recurso). `Favorite.findOne({ _id: req.params.id, userId: req.user.id })` — busca o documento **filtrando também pelo dono**. Isso é crucial: se outro usuário tentar editar o ID de alguém via `/favorites/<id-de-outro-usuario>`, a busca retorna `null` (porque o `userId` não bate) e cai no **404** — sem vaziar a informação de que o recurso existe mas pertence a outra pessoa. Depois de encontrar, sobrescreve cada campo manualmente e chama `.save()` (que aciona validações do schema novamente).

```
// PATCH /favorites/:id - partial update (only provided fields are changed)
router.patch('/:id', auth, async (req, res) => {
  const allowed = ['name', 'origin', 'destination', 'places'];
  const updates = Object.fromEntries(
    Object.entries(req.body).filter(([k]) => allowed.includes(k))
  );
});
```

PATCH = atualização **parcial**. `allowed` é a "whitelist" de campos que podem ser alterados — qualquer outra propriedade no `req.body` (ex: tentar mudar `userId` diretamente) é **ignorada**. `Object.entries(req.body)` transforma `{a:1, b:2}` em `[[ 'a', 1 ], [ 'b', 2 ] ]`; `.filter(([k])`

=> `allowed.includes(k)` mantém só os pares cuja chave está na whitelist;

`Object.fromEntries(...)` reconstrói o objeto a partir do array filtrado.

```
if (Object.keys(updates).length === 0) {
  return res.status(400).json({ error: 'No valid fields to update' });
}
if ('places' in req.body) {
  updates.places = normalizePlaces(req.body.places);
}
```

Se depois da filtragem não sobrou nenhum campo válido, **400** — evita um `findOneAndUpdate` "vazio" que não faz nada. Se `places` foi enviado, passa pela normalização antes de ir para `updates`.

```
try {
  const fav = await Favorite.findOneAndUpdate(
    { _id: req.params.id, userId: req.user.id },
    updates,
    { new: true, runValidators: true }
  );
  if (!fav) return res.status(404).json({ error: 'Favorite not found' });
  res.json(fav);
} catch (err) {
  res.status(500).json({ error: err.message });
}
});
```

`findOneAndUpdate(filtro, mudanças, opções)` — encontra e atualiza em uma única operação atômica.

- `{ new: true }` — retorna o documento **depois** da atualização (por padrão o Mongoose retornaria a versão antiga).
- `{ runValidators: true }` — sem essa opção, o `findOneAndUpdate` **não roda** as validações do schema (`required`, `minlength` etc.) por padrão — é um comportamento "armadilha" comum do Mongoose. Aqui garantimos que as regras continuam valendo mesmo numa atualização parcial.

Se o filtro não encontrar nada (ID não existe ou pertence a outro usuário), `fav` é `null` → **404**.

```
// DELETE /favorites/:id — remove a favorite
router.delete('/:id', auth, async (req, res) => {
  try {
    const fav = await Favorite.findOneAndDelete({ _id: req.params.id, userId: req.user.id });
```

```

    if (!fav) return res.status(404).json({ error: 'Favorite not found' });
    res.status(204).send();
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;

```

Mesmo padrão de filtro `{ _id, userId }` para garantir isolamento entre usuários. **204 No Content** — código correto para "deletado com sucesso, sem corpo de resposta". `.send()` sem argumento envia uma resposta vazia (chamar `.json()` com 204 seria tecnicamente incorreto, pois 204 não deve ter corpo).

## 7. routes/apiKeys.js

```

const router = require('express').Router();
const crypto = require('crypto');
const ApiKey = require('../models/ApiKey');
const auth = require('../middleware/auth');

```

`crypto` é o módulo nativo do Node para operações criptográficas — usado aqui só para gerar bytes aleatórios seguros.

```

// POST /api-keys - generate a new key (requires login)
router.post('/', auth, async (req, res) => {
  const { name } = req.body;
  if (!name) return res.status(400).json({ error: 'name is required' });

  const key = 'ptv_' + crypto.randomBytes(24).toString('hex');

```

Exige JWT ( `auth` ) — só usuários logados podem gerar API keys. `name` é obrigatório (apelido da chave). `crypto.randomBytes(24)` gera 24 bytes **criptograficamente aleatórios** (não é `Math.random()`, que não é seguro para isso). `.toString('hex')` converte os bytes para uma string hexadecimal de 48 caracteres. O prefixo `'ptv_'` é só uma convenção visual (facilita reconhecer o tipo de chave em logs, ex: como o GitHub faz com `ghp_...`).

```

try {
  const record = await ApiKey.create({ key, name, userId: req.user.id });

```

```

    res.status(201).json({ id: record._id, name: record.name, key: record.key, createdAt
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

Salva no banco associando ao usuário logado. Retorna **201** com a chave **completa** — esta é a única vez que ela aparece por inteiro na resposta da API.

```

// GET /api-keys - list your keys (key value is masked after creation)
router.get('/', auth, async (req, res) => {
  try {
    const keys = await ApiKey.find({ userId: req.user.id }).sort('-createdAt');
    res.json(keys.map(k => ({
      id: k._id,
      name: k.name,
      key: k.key.slice(0, 10) + '.....',
      active: k.active,
      createdAt: k.createdAt
    })));
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

Lista as chaves do usuário. `k.key.slice(0, 10)` pega só os 10 primeiros caracteres (ex: `ptv_a3f8b2`) e concatena com `.....` (16 bullets) — mascaramento visual. Isso permite ao usuário **identificar** qual chave é qual (pelo prefixo + nome), sem expor o valor completo de novo.

```

// DELETE /api-keys/:id - revoke a key
router.delete('/:id', auth, async (req, res) => {
  try {
    const record = await ApiKey.findOneAndDelete({ _id: req.params.id, userId: req.user.id });
    if (!record) return res.status(404).json({ error: 'API key not found' });
    res.status(204).send();
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;

```



`:id` aqui é o `_id` do **documento MongoDB** (não o valor da `key` em si — ponto de confusão comum). Mesmo padrão `{ _id, userId }` para garantir que só o dono pode revogar. **204** em caso de sucesso.

## 8. services/ptvSign.js

```
const crypto = require('crypto');

const BASE_URL = 'https://timetableapi.ptv.vic.gov.au';
```

Módulo nativo `crypto` (mesmo do `apiKeys.js`, mas usado aqui para HMAC em vez de bytes aleatórios). `BASE_URL` é o domínio fixo da PTV Timetable API.

```
function buildUrl(path, params = {}) {
  const devId = process.env.PTV_DEV_ID;
  const apiKey = process.env.PTV_API_KEY;
```

`path` é o caminho do endpoint (ex: `/v3/departures/route_type/0/stop/1071`). `params` são query params extras (ex: `{ max_results: 10 }`), com `{}` como padrão. `devId` e `apiKey` vêm do `.env` — credenciais fornecidas pela PTV ao se registrar como desenvolvedor.

```
const query = new URLSearchParams({ ...params, devid: devId }).toString();
const rawToSign = `${path}?${query}`;
```

`URLSearchParams` monta uma query string a partir de um objeto, cuidando automaticamente de URL-encoding (espaços viram `%20` etc). `{ ...params, devid: devId }` — espalha os params recebidos e adiciona `devid` (toda chamada à PTV precisa identificar o dev). `.toString()` produz algo como `"max_results=10&devid=12345"`. `rawToSign` é a string completa que será assinada: `"/v3/departures/route_type/0/stop/1071?max_results=10&devid=12345"`.

```
const signature = crypto
  .createHmac('sha1', apiKey)
  .update(rawToSign)
  .digest('hex')
  .toUpperCase();
```

**HMAC-SHA1** (Hash-based Message Authentication Code): um algoritmo que combina uma chave secreta ( `apiKey` ) com os dados ( `rawToSign` ) para gerar uma "assinatura" — uma sequência de

caracteres que só pode ser gerada por quem possui a chave.

- `crypto.createHmac('sha1', apiKey)` — cria o objeto HMAC usando SHA-1 como função de hash, com `apiKey` como segredo.
- `.update(rawToSign)` — alimenta os dados a serem assinados.
- `.digest('hex')` — finaliza o cálculo e retorna o resultado em hexadecimal.
- `.toUpperCase()` — a PTV exige a assinatura em **maiúsculas**.

```
return `${BASE_URL}${rawToSign}&signature=${signature}`;  
}  
  
module.exports = { buildUrl };
```

Monta a URL final, anexando `&signature=<hash>` no fim. Essa é a URL que será chamada com `axios.get(...)`. **Por que isso garante segurança?** Se alguém interceptar essa URL e tentar mudar `stop_id` para outro valor, a `signature` não vai mais bater com o novo `path / query` — a PTV recalcula o HMAC do lado dela (com a mesma `apiKey`, que só ela e nós conhecemos) e rejeita se não bater.

---

## 9. services/ptv.js

---

```
const axios = require('axios');  
const { buildUrl } = require('./ptvSign');  
  
async function getStop(stopId, routeType = 0) {  
  const url = buildUrl(`/v3/stops/${stopId}/route_type/${routeType}`);  
  const res = await axios.get(url);  
  return res.data.stop;  
}
```

Monta a URL assinada para o endpoint `/v3/stops/{stopId}/route_type/{routeType}` (detalhes de uma parada específica). `axios.get(url)` faz a requisição HTTP GET. `res.data` é o corpo da resposta JSON; `.stop` é a chave onde a PTV coloca os dados da parada (ex: `{ stop: { stop_id, stop_name, stop_suburb, ... } }`).

```
async function getDepartures(stopId, routeType = 0) {  
  const url = buildUrl(`/v3/departures/route_type/${routeType}/stop/${stopId}`);  
  const res = await axios.get(url);
```

```
    return res.data.departures;
  }
```

Endpoint `/v3/departures/route_type/{routeType}/stop/{stopId}` — lista de próximas partidas daquela parada. Retorna `res.data.departures` (array).

```
async function getRoute(routeId) {
  const url = buildUrl(`/v3/routes/${routeId}`);
  const res = await axios.get(url);
  return res.data.route;
}

module.exports = { getStop, getDepartures, getRoute };
```

Endpoint `/v3/routes/{routeId}` — detalhes de uma linha (nome, número, tipo). Cada `departure` retornado por `getDepartures` tem um `route_id`; essa função traduz esse ID em informações legíveis (ex: "Sandringham Line"). As três funções são exportadas e usadas em `server.js` na rota `/board/:stop_id`.

---

## 10. services/ptvLeg.js

---

```
const axios = require('axios');
const { buildUrl } = require('./ptvSign');

const routeCache = new Map();
```

`routeCache` é um `Map` (estrutura chave→valor) **em memória do processo** — não persiste no banco, vive enquanto o servidor está rodando. Guarda `route_id → { route_name, route_number }` para evitar chamar a PTV repetidamente pela mesma rota.

```
async function searchStop(name, routeType) {
  const term = name.split('/')[0].trim();
  const url = buildUrl(`/v3/search/${encodeURIComponent(term)}`);
  const res = await axios.get(url);
  const stops = res.data.stops || [];
  const stop = stops.find(s => s.route_type === routeType) || stops[0] || null;
  console.log(`[ptv] search "${term}" →`, stop ? `"${stop.stop_name}" id=${stop.stop_id}`
```

```

    return stop;
  }

```

- `name` vem do Google Routes, ex: `"Princes St/Fitzroy St"`. `.split('/')[0]` pega só `"Princes St"` — a PTV não reconhece o nome completo com barra.
- `.trim()` remove espaços extras.
- `buildUrl('/v3/search/...')` — endpoint de busca textual de paradas/rotas. `encodeURIComponent` garante que espaços e caracteres especiais no nome virem `%20` etc. e não quebrem a URL.
- `res.data.stops || []` — se a PTV não retornar `stops`, usa array vazio.
- `stops.find(s => s.route_type === routeType)` — a busca pode retornar paradas de tipos diferentes (ex: "Flinders Street" tem trem E tram); priorizamos a que bate com o `routeType` esperado (0=trem, 1=tram, 2=ônibus). Se nenhuma bater, `|| stops[0]` pega a primeira disponível. Se a lista estiver vazia, `|| null`.
- `console.log` ajuda a debugar no terminal do servidor quando uma parada "não bate" com o esperado.

```

async function getDepartures(stopId, routeType) {
  const url = buildUrl(`/v3/departures/route_type/${routeType}/stop/${stopId}`, {
    max_results: 10
  });
  const res = await axios.get(url);
  return res.data.departures || [];
}

```

Mesmo endpoint do `services/ptv.js`, mas pedindo `max_results: 10` (precisamos de margem para depois filtrar e ordenar) e já tratando ausência de `departures`.

```

async function getRouteInfo(routeId) {
  if (routeCache.has(routeId)) return routeCache.get(routeId);
  try {
    const url = buildUrl(`/v3/routes/${routeId}`);
    const res = await axios.get(url);
    const info = {
      route_name: res.data.route?.route_name || '',
      route_number: res.data.route?.route_number || ''
    };
    routeCache.set(routeId, info);
    return info;
  } catch {
    return { route_name: '', route_number: '' };
  }
}

```

```
}  
}
```

- `if (routeCache.has(routeId)) return routeCache.get(routeId)` — checa o cache primeiro; se já buscamos essa rota antes (em qualquer requisição anterior), retorna direto sem chamar a PTV de novo.
- Caso contrário, busca via `buildUrl('/v3/routes/{routeId}')`.  
`res.data.route?.route_name || ''` — `?.` evita erro se `route` vier `undefined`; `|| ''` garante string vazia em vez de `undefined`.
- `routeCache.set(routeId, info)` — grava no cache para a próxima vez.
- `catch { return {...vazio} }` — se der erro (ex: `route_id` inválido), não derruba a requisição inteira — retorna campos vazios, e **não** salva no cache (assim uma falha temporária pode ser tentada de novo depois).

```
// origin      - stop name from Google Routes (e.g. "Princes St/Fitzroy St")  
// routeType   - 0=train, 1=tram, 2=bus  
// lineShort   - route number from Google (e.g. "5", "78")  
// fromTime    - ISO string: show departures from this time onwards (defaults to now)  
async function getPtvLeg(origin, routeType = 0, lineShort = '', fromTime = null) {  
  const originStop = await searchStop(origin, routeType);  
  if (!originStop) return { found: false, error: `Stop not found: "${origin}"` };
```

Comentários documentam os parâmetros (úteis porque vêm de fontes diferentes — Google vs nosso código). Se `searchStop` não encontrar nada, retorna imediatamente `{ found: false, error: '...' }` — esse é o objeto que o `server.js` checa em `result.error` na rota `/ptv-leg`.

```
const departures = await getDepartures(originStop.stop_id, routeType);  
  
// Fetch route info for every unique route_id in one parallel batch  
const uniqueRouteIds = [...new Set(departures.map(d => d.route_id))];  
await Promise.all(uniqueRouteIds.map(id => getRouteInfo(id)));
```

Busca as próximas 10 partidas dessa parada. `departures.map(d => d.route_id)` extrai todos os `route_id` (pode ter repetidos — várias partidas da mesma linha). `new Set(...)` remove duplicatas; `[...Set]` converte de volta para array. `Promise.all(uniqueRouteIds.map(id => getRouteInfo(id)))` — busca info de **cada rota única em paralelo** (não sequencialmente) e popula o `routeCache` antes do próximo passo.

```
const refMs = fromTime ? new Date(fromTime).getTime() : Date.now();  
const nowMs = Date.now();
```

`refMs` — ponto de referência para "filtrar partidas a partir de quando". Se `fromTime` foi passado (horário de partida da viagem do Google), usa ele; senão usa "agora" ( `Date.now()` ). `nowMs` — sempre "agora", usado para calcular `mins_until` (quanto falta a partir do momento real, não do horário da viagem).

```
const results = departures
  .map(dep => {
    const scheduledMs = new Date(dep.scheduled_departure_utc).getTime();
    const minsUntil = Math.round((scheduledMs - nowMs) / 60000);
    const routeInfo = routeCache.get(dep.route_id) || { route_name: '', route_number: '' };
    return {
      route_id: dep.route_id,
      route_number: routeInfo.route_number,
      route_name: routeInfo.route_name,
      platform: dep.platform_number,
      scheduled_departure: dep.scheduled_departure_utc,
      mins_until: minsUntil,
      _scheduledMs: scheduledMs
    };
  })
```

Para cada partida: calcula `minsUntil` (minutos a partir de agora até a partida agendada) e busca a info da rota no cache (já populado no passo anterior). `_scheduledMs` é um campo **temporário** (prefixo `_` por convenção, indicando "interno") — usado só para filtrar/ordenar a seguir, será removido antes de retornar.

```
.filter(d => d._scheduledMs >= refMs - 60000)
.sort((a, b) => a.mins_until - b.mins_until)
.slice(0, 5)
.map(({ _scheduledMs, ...d }) => d);
```

- `.filter(d => d._scheduledMs >= refMs - 60000)` — mantém só partidas a partir de `refMs` menos 1 minuto de tolerância ( `60000` ms). A tolerância evita excluir uma partida que está acontecendo "agora mesmo" por um segundo de diferença.
- `.sort((a, b) => a.mins_until - b.mins_until)` — ordena crescente (a mais próxima primeiro). Função de comparação padrão do JS: se o resultado é negativo, `a` vem antes de `b`.
- `.slice(0, 5)` — pega só as 5 primeiras.
- `.map(({ _scheduledMs, ...d }) => d)` — destructuring que **separa** `_scheduledMs` do resto ( `...d` ) e retorna só `d` — remove o campo interno do resultado final.

```
console.log(`[ptv] "${originStop.stop_name}" (from ${fromTime || 'now'}) → ${results.length} results`)
```

```

return {
  found: true,
  origin_stop_id: originStop.stop_id,
  origin_stop: originStop.stop_name,
  primary_route_id: results[0]?.route_id || null,
  departures: results
};
}

module.exports = { getPtvLeg };

```

Log de debug mostrando um resumo legível ( `["5 in 3min", "5 in 18min", ...]` ). Retorna o objeto final: `found: true`, dados da parada encontrada, `primary_route_id` (route\_id da primeira/próxima partida — `results[0]?.route_id` usa `?.` caso `results` esteja vazio) e o array de até 5 `departures`.

## 11. services/journey.js

```

const axios = require('axios');

const ROUTES_API_URL = 'https://routes.googleapis.com/directions/v2:computeRoutes';

```

Endpoint do **Google Routes API v2** — a versão mais nova da API de rotas do Google (sucessora da antiga Directions API), que retorna dados em tempo real.

```

const FIELD_MASK = [
  'routes.legs.steps.transitDetails',
  'routes.legs.steps.travelMode',
  'routes.legs.duration',
  'routes.legs.distanceMeters',
  'routes.duration',
  'routes.distanceMeters'
].join(',');

```

A Routes API v2 **exige** um header `X-Goog-FieldMask` especificando quais campos você quer na resposta — sem ele, a API retorna erro. `.join(',')` transforma o array em string `"routes.legs.steps.transitDetails,routes.legs.steps.travelMode,..."`. Pedir só os campos necessários reduz o tamanho da resposta e o custo cobrado pelo Google (a Routes API cobra por SKU/campos solicitados).

```

async function getJourney(origin, destination) {
  const res = await axios.post(
    ROUTES_API_URL,
    {
      origin: { address: `${origin}, Melbourne, VIC, Australia` },
      destination: { address: `${destination}, Melbourne, VIC, Australia` },
      travelMode: 'TRANSIT',
      computeAlternativeRoutes: false
    },
  },

```

A API espera **POST** (não GET) com um corpo JSON. `origin` / `destination` são objetos `{ address: "..."` — concatenamos `", Melbourne, VIC, Australia"` ao texto digitado pelo usuário para reduzir ambiguidade (ex: "Town Hall" existe em várias cidades). `travelMode: 'TRANSIT'` pede rota de transporte público especificamente (vs. carro, a pé, bike). `computeAlternativeRoutes: false` — pedimos só **uma** rota (a melhor); `true` traria várias opções, mas aumentaria a complexidade do app.

```

{
  headers: {
    'Content-Type': 'application/json',
    'X-Goog-API-Key': process.env.GOOGLE_MAPS_API_KEY,
    'X-Goog-FieldMask': FIELD_MASK
  }
}
);

```

Terceiro argumento do `axios.post` é a configuração — aqui, os headers. `X-Goog-API-Key` é como a Routes API v2 recebe a chave (diferente de outras APIs do Google que usam `?key=` na query string). `X-Goog-FieldMask` é o filtro de campos definido acima.

```

const routes = res.data.routes;
if (!routes || routes.length === 0) return null;

const leg = routes[0].legs[0];

```

`res.data.routes` é um array (vazio se nenhuma rota foi encontrada — retornamos `null`, que o `server.js` transforma em **404**). `routes[0]` — primeira (e única, já que pedimos `computeAlternativeRoutes: false`) rota. `.legs[0]` — para uma viagem simples A→B sem waypoints intermediários, há sempre **um** "leg" no nível do Google (que internamente contém vários "steps" — cada step é um trecho a pé ou de transporte).



```

const legs = leg.steps
  .filter(s => s.transitDetails)
  .map(s => {
    const td = s.transitDetails;
    return {
      mode: td.transitLine?.vehicle?.type || s.travelMode,
      line: td.transitLine?.name || td.transitLine?.nameShort || '-',
      line_short: td.transitLine?.nameShort || '', // route number, e.g. "5", "78"
      headsign: td.headsign || '-',
      departure_stop: td.stopDetails?.departureStop?.name || '-',
      arrival_stop: td.stopDetails?.arrivalStop?.name || '-',
      departure_time: td.stopDetails?.departureTime || null,
      arrival_time: td.stopDetails?.arrivalTime || null,
      num_stops: td.stopCount || 0
    };
  });

```

- `leg.steps` é o array de **todos** os passos da viagem, incluindo trechos a pé (caminhar até a estação) e de transporte.
- `.filter(s => s.transitDetails)` — descarta os passos a pé (só `WALK` não tem `transitDetails`); ficamos só com trechos de trem/tram/ônibus — esses viram nossos `legs` (na nomenclatura do nosso app).
- Para cada step de transporte ( `s` ), `td = s.transitDetails` contém os detalhes:
  - `mode` — tipo de veículo (ex: `"TRAM"`, `"BUS"`, `"HEAVY_RAIL"`). Fallback para `s.travelMode` se `vehicle.type` não vier.
  - `line` — nome completo da linha (ex: "Sandringham") ou o nome curto, ou `'-'` se nada disponível.
  - `line_short` — número/código da linha (ex: "5", "78") — usado depois para casar com a numeração da PTV.
  - `headsign` — destino exibido no veículo (ex: "via City").
  - `departure_stop` / `arrival_stop` — nomes das paradas de embarque/desembarque, com `'-'` como fallback.
  - `departure_time` / `arrival_time` — horários **em tempo real** (ISO 8601) — o coração da detecção de atraso.
  - `num_stops` — quantidade de paradas no trecho ( `td.stopCount` ), `0` se ausente.

```

return {
  duration_seconds: Number(routes[0].duration?.replace('s', '')) || 0,
  distance_meters: routes[0].distanceMeters || 0,
  total_legs: legs.length,
  legs

```

```

    };
  }

  module.exports = { getJourney };

```

- `routes[0].duration` vem como string tipo `"1620s"` (segundos + sufixo "s", formato Protobuf `Duration`). `.replace('s', '')` remove o sufixo; `Number(...)` converte para número; `|| 0` cobre o caso de `duration` ausente.
- `distance_meters` — distância total em metros (já numérico no Google), com `|| 0` como fallback.
- `total_legs` / `legs` — contagem e array dos trechos de transporte processados acima.

## 12. services/alerts.js

```

// Compares Google Routes API real-time departure with PTV scheduled time.
// The difference is the delay – no external dependency required.

function formatTime(ms) {
  return new Date(ms).toLocaleTimeString('en-AU', {
    hour: '2-digit', minute: '2-digit',
    timeZone: 'Australia/Melbourne'
  });
}

```

Comentário explica a estratégia geral do arquivo. `formatTime` recebe um timestamp em milissegundos e formata como hora local de Melbourne ( `'en-AU'` , `'Australia/Melbourne'` — importante porque o servidor pode rodar em outro fuso horário, ex: nos EUA na nuvem). `hour: '2-digit', minute: '2-digit'` produz algo como `"09:05"` .

```

function getAlertsFromDeparture(googleDepartureTime, ptvScheduledTime) {
  if (!googleDepartureTime || !ptvScheduledTime) return [];
}

```

Se qualquer um dos dois horários não existir (ex: PTV não encontrou a parada), não há como comparar — retorna array vazio (sem alerta).

```

const googleMs = new Date(googleDepartureTime).getTime();
const ptvMs     = new Date(ptvScheduledTime).getTime();

```

```
if (isNaN(googleMs) || isNaN(ptvMs)) return [];
```

Converte ambas as strings ISO para milissegundos. `new Date('string inválida').getTime()` retorna `NaN` — `isNaN(...)` detecta isso e aborta com segurança (evita cálculos sem sentido com `NaN`).

```
// Skip if times are more than 60 min apart – likely different services
if (Math.abs(googleMs - ptvMs) > 60 * 60 * 1000) return [];
```

`Math.abs(...)` — valor absoluto da diferença (ignora sinal). `60 * 60 * 1000` = 3.600.000 ms = 1 hora. Se a diferença passa de 1 hora, provavelmente o Google e a PTV estão se referindo a **partidas diferentes** daquela linha (ex: o trem das 9h vs o das 10h) — comparar daria um "atraso" absurdo e enganoso, então ignoramos.

```
const delayMinutes = Math.round((googleMs - ptvMs) / 60000);

if (delayMinutes >= 2) {
  return [{
    type: 'delay',
    title: `Running ${delayMinutes} min late`,
    description: `Expected ${formatTime(googleMs)} · Scheduled ${formatTime(ptvMs)}`
  }];
}

return [];
}

module.exports = { getAlertsFromDeparture };
```

`delayMinutes = (tempo real Google) - (horário agendado PTV)`, em minutos, arredondado. Se positivo, o veículo está saindo **depois** do previsto = atraso. `>= 2` — usamos uma tolerância de 2 minutos: pequenas variações (1 minuto) são normais e não justificam mostrar um alerta visual ("ruído"). Se `delayMinutes < 2` (no horário, adiantado, ou atraso insignificante), retorna `[]` — sem alerta (o frontend então mostra "Running on time").

O objeto de alerta tem:

- `type: 'delay'` — usado pelo frontend para escolher o ícone/estilo (🚦 vermelho/amarelo vs ⚠️ genérico).
- `title` — mensagem curta e direta.
- `description` — mostra os dois horários formatados, para o usuário entender de onde veio o número.

## 13. services/commonAlerts.js

```
const axios = require('axios');
const { buildUrl } = require('./ptvSign');

// Maps route_type to PTV disruption categories
const CATEGORIES = {
  0: ['metro_train', 'regional_train'],
  1: ['metro_tram'],
  2: ['metro_bus', 'regional_bus', 'night_bus']
};
```

A PTV organiza disruptions (interrupções/avisos) por categorias de texto, não pelos mesmos números `route_type` (0/1/2) usados em outros endpoints. Este mapa traduz: trem ( `0` ) → `metro_train` + `regional_train` ; tram ( `1` ) → `metro_tram` ; ônibus ( `2` ) → três subcategorias de ônibus.

```
async function getCommonAlerts(routeType = 0) {
  const url = buildUrl('/v3/disruptions');
  const res = await axios.get(url);
  const disruptions = res.data.disruptions || {};
```

Endpoint `/v3/disruptions` retorna **todas** as disruptions de **toda a rede PTV**, agrupadas por categoria — `res.data.disruptions` é um objeto tipo `{ metro_train: [...], metro_tram: [...], ... }`. `|| {}` protege contra resposta vazia.

```
const categories = CATEGORIES[routeType] || CATEGORIES[0];
const alerts = [];

for (const cat of categories) {
  for (const d of (disruptions[cat] || [])) {
```

`CATEGORIES[routeType] || CATEGORIES[0]` — se `routeType` não for 0/1/2 (valor inesperado), usa trem como padrão. Loop duplo: para cada categoria relevante ( `cat` ), itera sobre as disruptions daquela categoria ( `disruptions[cat] || []` — array vazio se a PTV não retornou nada para essa categoria).

```
const isDelay = /delay|late/i.test(d.disruption_type || '') ||
  /delay|late/i.test(d.title || '');
```

```

    alerts.push({
      type: isDelay ? 'delay' : 'disruption',
      title: d.title,
      description: d.description || ''
    });
  }
}

return { alerts };
}

module.exports = { getCommonAlerts };

```

`/delay|late/i` — expressão regular que casa com "delay" ou "late" (case-insensitive, flag `i`), testada tanto no `disruption_type` quanto no `title` da PTV. Se bater em qualquer um, classificamos como `'delay'`; senão, `'disruption'` genérica (ex: obras, mudança de rota, cancelamento). `d.description || ''` evita `undefined` no JSON de saída. Retorna `{ alerts: [...] }` — usado pela rota `/common-alerts` no `server.js`.

## 14. services/places.js

```

const axios = require('axios');

const CATEGORY_MAP = {
  coffee: { label: 'Coffee', type: 'cafe', keyword: 'coffee' },
  restaurant: { label: 'Restaurant', type: 'restaurant' },
  food: { label: 'Food', type: 'restaurant', keyword: 'food' },
  bar: { label: 'Bar', type: 'bar' },
  'night club': { label: 'Night Club', type: 'night_club' },
  museum: { label: 'Museum', type: 'museum' },
  hotel: { label: 'Hotel', type: 'lodging' },
  lodging: { label: 'Lodging', type: 'lodging' },
  hospital: { label: 'Hospital', type: 'hospital' },
  'public toilet': { label: 'Public toilet', keyword: 'public toilet' }
};

```

Tabela de tradução: nossas categorias internas (chaves do app) → `type` / `keyword` que o **Google Places Nearby Search** entende. `type` é um valor controlado e oficial do Google (lista fixa de tipos, ex: `'cafe'`, `'restaurant'`, `'lodging'`); `keyword` é busca textual livre, usada quando não existe `type` exato (ex: "public toilet" não é um `type` do Google) ou para refinar (ex: "coffee"

usa `type: 'cafe'` + `keyword: 'coffee'` para filtrar cafeterias que realmente vendem café, não só padarias).

```
const DEFAULT_CATEGORIES = [  
  'coffee',  
  'restaurant',  
  'hotel',  
  'public toilet',  
  'hospital',  
  'bar',  
  'night club',  
  'museum'  
];
```

Categorias usadas quando o usuário não seleciona nenhuma (ou envia parâmetro inválido) — busca em todas.

```
const GOOGLE_FIND_PLACE_URL = 'https://maps.googleapis.com/maps/api/place/findplacefromt  
const GOOGLE_NEARBY_SEARCH_URL = 'https://maps.googleapis.com/maps/api/place/nearbysearch
```

Dois endpoints diferentes do Google Places API (legado, mas ainda funcional e mais simples que a versão "New"): "Find Place from Text" (geocodificar um endereço/nome em coordenadas) e "Nearby Search" (buscar lugares perto de um ponto).

```
async function geocodeDestination(destination, apiKey) {  
  const response = await axios.get(GOOGLE_FIND_PLACE_URL, {  
    params: {  
      input: destination,  
      inputtype: 'textquery',  
      fields: 'formatted_address,geometry,place_id',  
      key: apiKey,  
      language: 'en'  
    }  
  });  
  
  const result = response.data.candidates?.[0];  
  if (!result) return null;  
  return {  
    address: result.formatted_address,  
    location: result.geometry.location,  
    placeId: result.place_id  
  };  
}
```

```
};
}
```

- `input: destination` — texto livre (ex: "Melbourne Central").
- `inputtype: 'textquery'` — diz ao Google que `input` é texto (não um `place_id`).
- `fields: '...'` — restringe os campos retornados (reduz custo).
- `response.data.candidates?.[0]` — a API retorna `candidates` (array); pegamos o primeiro/melhor. `?.` evita erro se `candidates` for `undefined`.
- Se não houver candidato, `null` (o `server.js` transforma isso em **404**).
- Caso contrário, retorna endereço formatado, `location` ( `{ lat, lng }` ) e `place_id`.

```
function getDistanceMeters(a, b) {
  const toRad = degrees => degrees * Math.PI / 180;
  const lat1 = a.lat;
  const lon1 = a.lng;
  const lat2 = b.lat;
  const lon2 = b.lng;
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const rad = 6371000; // Earth radius in meters
  const sinLat = Math.sin(dLat / 2);
  const sinLon = Math.sin(dLon / 2);
  const hav = sinLat * sinLat + Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * sinLon *
  const c = 2 * Math.atan2(Math.sqrt(hav), Math.sqrt(1 - hav));
  return Math.round(rad * c);
}
```

Implementação da **fórmula de Haversine** — calcula a distância em linha reta entre dois pontos geográficos (lat/lng), considerando que a Terra é uma esfera (não um plano).

- `toRad` converte graus para radianos (funções trigonométricas do JS trabalham em radianos).
- `dLat` / `dLon` — diferenças de latitude/longitude em radianos.
- `rad = 6371000` — raio médio da Terra em metros.
- `hav` — o "haversine" da diferença angular: combina as diferenças de lat/lon ponderadas pelo cosseno das latitudes (corrige a distorção de longitude perto dos polos — não muito relevante em Melbourne, mas é a fórmula correta e genérica).
- `c = 2 * atan2(√hav, √(1-hav))` — converte o haversine de volta em ângulo central (em radianos).
- `rad * c` — arco = raio × ângulo = distância em metros. `Math.round` arredonda para metro inteiro.

```

async function nearbyPlaces({ location, category, apiKey, radius = 500, limit = 5 }) {
  const categoryMeta = CATEGORY_MAP[category];
  if (!categoryMeta) return { category, label: category, places: [] };

```

Recebe um objeto com destructuring (parâmetros nomeados). `radius = 500` (metros) e `limit = 5` são padrões. Se a categoria não existe no `CATEGORY_MAP`, retorna estrutura vazia em vez de erro — mantém o formato consistente para o frontend.

```

const params = {
  key: apiKey,
  location: `${location.lat},${location.lng}`,
  radius,
  language: 'en'
};

if (categoryMeta.type) params.type = categoryMeta.type;
if (categoryMeta.keyword) params.keyword = categoryMeta.keyword;
if (categoryMeta.type && categoryMeta.keyword && category === 'coffee') {
  params.keyword = categoryMeta.keyword;
}

```

Monta os parâmetros da requisição Nearby Search. `location` precisa ser uma string `"lat,lng"` (formato exigido pelo Google, diferente do objeto usado internamente). Adiciona `type` e/ou `keyword` condicionalmente, conforme definido no `CATEGORY_MAP` para aquela categoria.

A terceira condição ( `category === 'coffee'` ) é redundante — `params.keyword` já foi setado pela linha anterior ( `if (categoryMeta.keyword)` ) já que "coffee" tem `keyword: 'coffee'` no `CATEGORY_MAP`. Não causa bug, mas é código que poderia ser removido sem alterar o comportamento.

```

const response = await axios.get(GOOGLE_NEARBY_SEARCH_URL, { params });
const results = response.data.results || [];

```

Faz a chamada e extrai `results` (array de lugares encontrados, até 20 por padrão no Google).

```

const filtered = results
  .map(place => {
    const placeLocation = place.geometry?.location || null;
    const distance_meters = placeLocation ? getDistanceMeters(location, placeLocation)
    return {
      raw: place,

```



```

        placeLocation,
        distance_meters
    });
  })
  .filter(item => item.distance_meters !== null && item.distance_meters <= radius)
  .sort((a, b) => a.distance_meters - b.distance_meters)
  .slice(0, limit);

```

- Para cada lugar bruto do Google, extrai a localização ( `place.geometry?.location` ) e calcula a distância real até o destino com `getDistanceMeters` .
- `.filter(...)` — descarta lugares sem coordenadas válidas OU que estejam **além** do raio pedido. Isso é necessário porque o Google às vezes retorna resultados ligeiramente fora do `radius` solicitado (a busca do Google é "aproximada").
- `.sort(...)` — ordena do mais perto para o mais longe.
- `.slice(0, limit)` — pega só os `limit` (padrão 5) mais próximos.

```

return {
  category,
  label: categoryMeta.label,
  places: filtered.map(item => ({
    category,
    name: item.raw.name,
    address: item.raw.vicinity || item.raw.formatted_address || null,
    place_id: item.raw.place_id,
    location: item.placeLocation,
    rating: item.raw.rating ?? null,
    user_ratings_total: item.raw.user_ratings_total ?? null,
    types: item.raw.types || [],
    open_now: item.raw.opening_hours?.open_now ?? null,
    business_status: item.raw.business_status || null,
    distance_meters: item.distance_meters,
    walking_minutes: Math.max(1, Math.round(item.distance_meters / 83.33))
  )))
};
}

```

Monta o resultado final por lugar, extraindo apenas os campos relevantes do objeto bruto do Google ( `item.raw` ):

- `address` — `vicinity` (endereço resumido, ex: "123 Collins St") ou `formatted_address` como fallback, ou `null` .
- `rating` / `user_ratings_total` — usa `?? null` (nullish) porque `0` seria um valor válido de rating (embora raro) e não deveria virar `null` .

- `open_now` — `item.raw.opening_hours?.open_now ?? null` — `?.` evita erro se `opening_hours` não existir; resultado pode ser `true`, `false` ou `null` (desconhecido).
- `walking_minutes: Math.max(1, Math.round(distance / 83.33))` — `83.33` m/min  $\approx$  5 km/h (velocidade média de caminhada). `Math.max(1, ...)` garante um mínimo de "1 min" mesmo para distâncias muito curtas (evita mostrar "0 min walk").

```
function normalizeCategories(input) {
  if (!input) return DEFAULT_CATEGORIES;
  const segments = input.split(',').map(s => s.trim().toLowerCase()).filter(Boolean);
  const normalized = [];
  for (const value of segments) {
    if (CATEGORY_MAP[value]) normalized.push(value);
  }
  return normalized.length ? normalized : DEFAULT_CATEGORIES;
}
```

Recebe a string de categorias da query string (ex: `"coffee,Hotel, museum"`). Se vazio/ `undefined`, usa `DEFAULT_CATEGORIES`. Senão: separa por vírgula, normaliza (trim + lowercase), remove vazios. Para cada valor, só mantém se existir no `CATEGORY_MAP` (ignora categorias inválidas/desconhecidas silenciosamente). Se depois de tudo a lista ficar vazia (todas inválidas), volta para `DEFAULT_CATEGORIES` — nunca retorna lista vazia.

```
async function getTargetPlaces(destination, categories, apiKey) {
  const geo = await geocodeDestination(destination, apiKey);
  if (!geo) return null;

  const normalizedCategories = normalizeCategories(categories);
  const placesByCategory = await Promise.all(
    normalizedCategories.map(category => nearbyPlaces({ location: geo.location, category
  });

  return {
    destination: geo.address,
    location: geo.location,
    categories: placesByCategory
  };
}

module.exports = {
  CATEGORY_MAP,
  normalizeCategories,
```

```
getTargetPlaces  
};
```

Função principal exportada (chamada pelo `server.js` em `/target-places`):

1. Geocodifica o destino → coordenadas. Se falhar, `null`.
2. Normaliza a lista de categorias pedidas.
3. `Promise.all(...)` — busca lugares de **todas as categorias em paralelo** (uma chamada Nearby Search por categoria, simultaneamente, em vez de uma de cada vez).
4. Retorna endereço formatado do destino, suas coordenadas, e o array de resultados por categoria.

`CATEGORY_MAP` e `normalizeCategories` também são exportados — não usados fora deste arquivo atualmente, mas disponíveis para testes ou extensões.

---

## 15. public/index.html — `<script>` (Frontend principal)

---

### Localização e geocodificação reversa

---

```
async function useMyLocation() {  
  if (!navigator.geolocation) {  
    alert('Geolocation is not supported by your browser.');    return;  
  }  
}
```

`navigator.geolocation` é a API nativa do **browser** para acessar GPS/localização. Se o browser não suportar (raro hoje em dia), avisa e sai.

```
const btn = document.getElementById('locBtn');  
const input = document.getElementById('origin');  
btn.classList.add('loading');  
btn.disabled = true;  
input.placeholder = 'Getting location...';
```

Pega referências ao botão de localização e ao campo de origem. Adiciona classe CSS `loading` (provavelmente mostra um spinner), desabilita o botão (evita cliques duplicados) e muda o

placeholder para dar feedback visual imediato.

```
navigator.geolocation.getCurrentPosition(
  async ({ coords }) => {
    try {
      const res = await fetch(`/reverse-geocode?lat=${coords.latitude}&lng=${coords.longitude}`);
      const data = await res.json();
      if (res.ok) {
        input.value = data.address;
        input.placeholder = 'Origin station or suburb';
      } else {
        input.placeholder = 'Could not get address';
      }
    } catch {
      input.placeholder = 'Request failed';
    } finally {
      btn.classList.remove('loading');
      btn.disabled = false;
    }
  },
```

`getCurrentPosition(successCallback, errorCallback, options)` — pede permissão de localização ao usuário (popup do browser). Se concedida, `successCallback` recebe `{ coords: { latitude, longitude, ... } }`. Chamamos nosso backend `/reverse-geocode` (que por sua vez chama o Google Geocoding) passando lat/lng. Se `res.ok` (status 2xx), preenche o campo `origin` com o endereço retornado. `finally` sempre roda — remove o estado de loading independente de sucesso/erro.

```
() => {
  input.placeholder = 'Location access denied';
  btn.classList.remove('loading');
  btn.disabled = false;
},
{ timeout: 8000 }
);
}
```

`errorCallback` — roda se o usuário **negar** a permissão (ou outro erro de geolocalização). `{ timeout: 8000 }` — desiste depois de 8 segundos esperando a localização.

## Navegação / sessão (login)

---

```
const _token = localStorage.getItem('ptv_token');
const _user = JSON.parse(localStorage.getItem('ptv_user') || 'null');
```

`localStorage` persiste dados no browser entre sessões (sobrevive a fechar a aba/navegador). `_token` é o JWT salvo no login/registro. `_user` é o objeto `{ id, email }` salvo como JSON string — `JSON.parse(... || 'null')` converte de volta para objeto, ou `null` se não existir nada salvo (a string `'null'` literal é o que `JSON.parse` recebe nesse caso, e produz o valor `null`).

```
function updateNav() {
  const navAuth = document.getElementById('navAuth');
  if (_user && _token) {
    navAuth.innerHTML = `
      <span class="nav-user">${_user.email}</span>
      <button class="nav-btn-ghost" onclick="logout()">Logout</button>`;
  } else {
    navAuth.innerHTML = `
      <a href="/login.html" class="nav-btn-ghost">Login</a>
      <a href="/register.html" class="nav-btn-accent" style="margin-left:6px">Register</a>
    `;
  }
}
```

Renderiza a área de navegação no topo: se há usuário logado, mostra o email + botão "Logout"; senão, mostra links "Login"/"Register". `innerHTML` substitui o conteúdo do elemento por essa string HTML.

```
function logout() {
  localStorage.removeItem('ptv_token');
  localStorage.removeItem('ptv_user');
  window.location.reload();
}

updateNav();
```

`logout()` apaga as duas chaves do `localStorage` e recarrega a página (`window.location.reload()`) — ao recarregar, `_token` / `_user` voltam `null`, e `updateNav()` (chamado de novo na re-execução do script) mostra os links de login. A chamada `updateNav()` no nível raiz roda assim que o script carrega, configurando a navbar corretamente desde o início.

## Salvar jornada (favoritos)

```
let _lastOrigin = '', _lastDestination = '';

async function saveJourney() {
  if (!_token) { window.location.href = '/login.html'; return; }
}
```

Variáveis globais que guardam a última busca feita (preenchidas dentro de `search()`, mais abaixo). Se o usuário não está logado (`!_token`), redireciona para a tela de login antes de tentar salvar.

```
const name = document.getElementById('saveNameInput').value.trim();
if (!name) { document.getElementById('saveNameInput').focus(); return; }
const btn = document.getElementById('saveBannerBtn');
const successEl = document.getElementById('saveSuccess');
const saveErrorEl = document.getElementById('saveError');
btn.disabled = true;
saveErrorEl.style.display = 'none';
saveErrorEl.textContent = '';
```

Lê o nome digitado pelo usuário para a rota. Se vazio, foca o campo (chama atenção visualmente) e cancela. Desabilita o botão de salvar e limpa qualquer mensagem de erro anterior.

```
try {
  const res = await fetch('/favorites', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${_token}` },
    body: JSON.stringify({ name, origin: _lastOrigin, destination: _lastDestination, poi: _selectedPoiPlaces });
  });
}
```

`POST /favorites` com o JWT no header `Authorization: Bearer <token>` (exigido pelo middleware `auth` no backend). O corpo inclui o nome digitado, a última origem/destino pesquisados, e quaisquer POIs já selecionados pelo usuário (`getSelectedPoiPlaces()`, definida mais abaixo).

```
if (res.status === 401) { window.location.href = '/login.html'; return; }
if (res.ok) {
  successEl.style.display = 'inline';
  saveErrorEl.style.display = 'none';
  btn.style.display = 'none';
  setTimeout(() => { successEl.style.display = 'none'; btn.style.display = ''; }, 3000);
} else {
  const data = await res.json().catch(() => ({}));
  saveErrorEl.textContent = data.error || 'Could not save journey';
}
```

```

        saveErrorEl.style.display = 'block';
    }
  } finally {
    btn.disabled = false;
  }
}

```

- `401` — token expirou/inválido → manda para login.
- Sucesso ( `res.ok` , status 2xx) — mostra mensagem "salvo" ( `successEl` ), esconde o botão por 3 segundos ( `setTimeout` ) e depois restaura o estado original.
- Erro (4xx/5xx que não seja 401) — `res.json().catch(() => ({}))` tenta ler o corpo de erro; se falhar (corpo não é JSON válido), usa objeto vazio `{}` . Mostra `data.error` ou uma mensagem genérica.
- `finally` — sempre reabilita o botão, com sucesso ou erro.

## Constantes de exibição e formatação

```

const ICONS = {
  HEAVY_RAIL: '🚆', COMMUTER_TRAIN: '🚆', RAIL: '🚆',
  TRAM: '🚊', BUS: '🚌', SUBWAY: '🚇', FERRY: '🚢'
};

const MODE_LABEL = {
  HEAVY_RAIL: 'Train', COMMUTER_TRAIN: 'Train', RAIL: 'Train',
  TRAM: 'Tram', BUS: 'Bus', SUBWAY: 'Metro', FERRY: 'Ferry'
};

const MODE_COLOR = {
  HEAVY_RAIL: '#1565c0', COMMUTER_TRAIN: '#1565c0', RAIL: '#1565c0',
  TRAM: '#2e7d32', BUS: '#c62828', SUBWAY: '#6a1b9a', FERRY: '#00695c'
};

```

Três "tabelas de lookup" que traduzem os valores `mode` retornados pelo Google ( `"TRAM"` , `"BUS"` etc — vindos de `services/journey.js` ) em: emoji, rótulo legível em inglês, e cor hexadecimal para o "chip" visual de cada trecho.

```

function formatTime(iso) {
  if (!iso) return '-';
  return new Date(iso).toLocaleTimeString('en-AU', {
    hour: '2-digit', minute: '2-digit',
    timeZone: 'Australia/Melbourne'
  });
}

```

```
});  
}
```

Converte uma string ISO de data/hora em horário local de Melbourne formatado ( "09:05" ). `if (!iso) return '-'` — trata `null` / `undefined` / string vazia mostrando um traço em vez de "Invalid Date".

```
function formatDuration(s) {  
  const m = Math.round(s / 60);  
  return m < 60 ? `${m} min` : `${Math.floor(m/60)}h ${m % 60}min`;  
}
```

Recebe segundos, converte para minutos. Se menos de 1 hora, mostra "35 min". Senão, mostra horas e minutos restantes: `Math.floor(m/60)` = horas inteiras, `m % 60` = minutos restantes (resto da divisão) → "1h 15min".

```
function formatDistance(m) {  
  return m >= 1000 ? `${(m / 1000).toFixed(1)} km` : `${m} m`;  
}
```

Se a distância for  $\geq 1000$  metros, mostra em km com 1 casa decimal ( "3.4 km" ); senão, em metros inteiros ( "650 m" ).

```
function esc(str) {  
  return String(str)  
    .replace(/&/g, '&amp;')  
    .replace(/"/g, '&quot;')  
    .replace(/'/g, '&#39;')  
    .replace(/</g, '&lt;')  
    .replace(/>/g, '&gt;');  
}
```

Função de **escape de HTML** — converte caracteres especiais em suas entidades HTML correspondentes. Usada sempre que inserimos texto vindo de uma API externa (nomes de lugares, endereços) dentro de `innerHTML`. **Por que importa:** sem isso, se um lugar se chamasse `<script>alert(1)</script>` (nome malicioso/exótico vindo do Google Places), o browser executaria esse script — uma vulnerabilidade de **XSS (Cross-Site Scripting)**. `String(str)` garante que funciona mesmo se `str` não for string (ex: `undefined` → "undefined"). A ordem dos `.replace` importa: `&` é trocado primeiro, senão os `&amp;` / `&quot;`; etc. gerados depois seriam re-escapados incorretamente.



## Google Maps

```
let map, directionsService, directionsRenderer;  
let infoWindows = [];  
let poiMapMarkers = [], poiMapInfoWindow;  
let mapsApiPromise = null;
```

Variáveis globais (estado do mapa):

- `map` — instância do mapa Google.
- `directionsService` / `directionsRenderer` — objetos da API de Directions do Google Maps JS (calculam e desenharam a rota visualmente).
- `infoWindows` — array dos popups de info de cada parada da rota (para poder fechá-los todos de uma vez).
- `poiMapMarkers` / `poiMapInfoWindow` — marcadores e popup dos pontos de interesse (POIs).
- `mapsApiPromise` — cache da Promise de carregamento do script do Google Maps (evita carregar o script JS duas vezes).

```
async function loadMapsApi() {  
  if (mapsApiPromise) return mapsApiPromise;  
  
  mapsApiPromise = (async () => {  
    const cfg = await fetch('/config').then(r => r.json());
```

Se já existe uma Promise em andamento/concluída, retorna ela (evita múltiplas inicializações simultâneas se a função for chamada várias vezes). Senão, busca `/config` no nosso backend para pegar a `mapsApiKey` (a chave nunca fica hardcoded no HTML).

```
await new Promise((resolve, reject) => {  
  const script = document.createElement('script');  
  script.src = `https://maps.googleapis.com/maps/api/js?key=${cfg.mapsApiKey}`;  
  script.onload = resolve;  
  script.onerror = reject;  
  document.head.appendChild(script);  
});
```

Carrega o **SDK do Google Maps JavaScript** dinamicamente: cria uma tag `<script src="...">` e injeta no `<head>`. `new Promise((resolve, reject) => {...})` envolve esse carregamento assíncrono baseado em eventos (`onload` / `onerror`) numa Promise, para podermos usar `await`. Só depois que o script carrega (e a variável global `google` fica disponível) o código continua.

```

map = new google.maps.Map(document.getElementById('map'), {
  center: { lat: -37.8136, lng: 144.9631 },
  zoom: 12,
  mapTypeControl: false,
  streetViewControl: false,
  fullscreenControl: false,
  styles: [
    { featureType: 'poi', stylers: [{ visibility: 'off' }] },
    { featureType: 'transit.station', stylers: [{ visibility: 'simplified' }] }
  ]
});
directionsService = new google.maps.DirectionsService();
directionsRenderer = new google.maps.DirectionsRenderer({ map, suppressMarkers: true }
})();

return mapsApiPromise;
}

```

Cria o mapa centrado em Melbourne ( `-37.8136, 144.9631` ), zoom 12. Desabilita controles de tipo de mapa, Street View e tela cheia (interface mais limpa). `styles` customiza a aparência: esconde os POIs padrão do Google ( `featureType: 'poi' → visibility: 'off'` ) — já que temos nossos próprios marcadores de POI — e simplifica os ícones de estações de transporte (menos poluição visual). `directionsService` / `directionsRenderer` são inicializados; `suppressMarkers: true` desliga os marcadores padrão A/B do Google (usamos marcadores customizados). A IIFE assíncrona `(async () => {...})();` é atribuída a `mapsApiPromise` — qualquer chamador que faça `await loadMapsApi()` espera tudo isso terminar.

```

function clearPoiMarkers() {
  poiMapMarkers.forEach(marker => marker.setMap(null));
  poiMapMarkers = [];
  if (poiMapInfoWindow) poiMapInfoWindow.close();
}

```

Remove todos os marcadores de POI do mapa ( `marker.setMap(null)` é como o Google Maps API "remove" um marcador) e limpa o array. Fecha o popup de info se estiver aberto. Chamado antes de desenhar uma nova busca de POIs (evita acumular marcadores de buscas anteriores).

```

function showPoiOnExistingMap(data) {
  if (!map) return;

  const places = (data.categories || [])

```

```
.flatMap(category => (category.places || []).map(place => ({ ...place, category: cat  
.filter(place => place.location && place.location.lat != null && place.location.lng
```

`if (!map) return` — proteção: se o mapa ainda não carregou, não faz nada. `data.categories` é o array retornado por `/target-places` — cada categoria tem seu próprio array `places`. `.flatMap(...)` "achata" essa estrutura aninhada em uma única lista de lugares — para cada `place`, cria uma cópia (`{ ...place, category: ... }`) garantindo que o campo `category` está presente (usa o da categoria-pai se o lugar não tiver). `.filter(...)` descarta lugares sem coordenadas válidas (não dá pra colocar marcador sem lat/lng).

```
clearPoiMarkers();  
  
if (!places.length) return;  
  
if (!poiMapInfoWindow) {  
  poiMapInfoWindow = new google.maps.InfoWindow();  
}  
  
const bounds = new google.maps.LatLngBounds();
```

Limpa marcadores antigos. Se não há lugares, para por aqui (mapa fica "limpo"). Cria o `InfoWindow` (popup) uma única vez e reutiliza (`if (!poiMapInfoWindow)` — lazy initialization). `LatLngBounds` é um "retângulo" geográfico que vai "crescer" para englobar todos os marcadores — usado depois para ajustar o zoom/centro do mapa automaticamente.

```
places.forEach((place, index) => {  
  const position = {  
    lat: Number(place.location.lat),  
    lng: Number(place.location.lng)  
  };  
  const marker = new google.maps.Marker({  
    position,  
    map,  
    label: {  
      text: String(index + 1),  
      color: 'white',  
      fontWeight: '700',  
      fontSize: '12px'  
    },  
    title: place.name || 'POI',  
    icon: {  
      path: google.maps.SymbolPath.CIRCLE,  
      scale: 10,
```

```

        fillColor: getPoiColor(place.category),
        fillOpacity: 1,
        strokeColor: '#001a52',
        strokeWeight: 2.5
      }
    });

```

Para cada lugar, cria um **Marker** (pino) no mapa:

- **position** — coordenadas (convertidas para **Number** por segurança, caso venham como string).
- **label** — número sequencial (1, 2, 3...) dentro do círculo, branco, negrito.
- **title** — tooltip ao passar o mouse (nome do lugar, ou "POI" se sem nome).
- **icon** — em vez de um pino padrão, desenha um **círculo customizado** ( **SymbolPath.CIRCLE** ) com **scale: 10** (tamanho), preenchido com a cor da categoria ( **getPoiColor** , definida mais abaixo) e borda azul-escura.

```

marker.addListener('click', () => {
  poiMapInfoWindow.setContent(`
    <div style="font-family:Inter,sans-serif;font-size:13px;line-height:1.5;max-width:
      <strong style="color:#1a202c;">${esc(place.name || 'Unnamed place')}</strong><
      <span style="color:#64748b;">${esc(place.category || 'Unknown category')}</span><
      <span style="color:#475569;">${esc(place.address || 'Address unavailable')}</span></div>`);
  poiMapInfoWindow.open(map, marker);
});

poiMapMarkers.push(marker);
bounds.extend(position);
});

```

Ao clicar no marcador, popula o **InfoWindow** com nome/categoria/endereço (todos passados por **esc()** — escape de HTML, já que vêm de dados externos do Google Places) e o abre ancorado naquele marcador. Adiciona o marcador ao array de controle e expande o **bounds** para incluir essa posição.

```

if (places.length === 1) {
  map.setCenter(bounds.getCenter());
  map.setZoom(Math.max(map.getZoom() || 14, 15));
} else {
  map.fitBounds(bounds, 48);
}

```

```
}  
}
```

Se há **só um** lugar, `fitBounds` deixaria o zoom artificialmente baixo (um único ponto não define uma "área") — então centralizamos nele e forçamos um zoom mínimo de 15 (`Math.max(zoomAtual, 15)`). Se há **múltiplos** lugares, `fitBounds(bounds, 48)` ajusta automaticamente centro e zoom para que todos os marcadores fiquem visíveis, com `48` pixels de margem (padding) nas bordas.

## Estado do mapa (rota vs vazio)

```
function setRouteMapState(hasRoute) {  
  const mapEl = document.getElementById('map');  
  const emptyEl = document.getElementById('mapEmpty');  
  if (!mapEl || !emptyEl) return;  
  
  if (hasRoute) {  
    mapEl.style.display = 'block';  
    emptyEl.style.display = 'none';  
    if (map) {  
      google.maps.event.trigger(map, 'resize');  
    }  
  } else {  
    mapEl.style.display = 'none';  
    emptyEl.style.display = 'flex';  
  }  
}
```

Alterna entre mostrar o mapa (`#map`) ou um placeholder vazio (`#mapEmpty`), provavelmente um texto tipo "Sem dados de mapa". `google.maps.event.trigger(map, 'resize')` — **truque necessário**: se o `<div>` do mapa estava com `display: none`, o Google Maps "não sabe" suas dimensões reais; ao tornar visível de novo, é preciso disparar manualmente o evento `resize` para o mapa recalcular seu tamanho e renderizar corretamente (sem isso, o mapa apareceria cinza/cortado).

## Desenhar a rota no mapa

```
function showRouteOnMap(origin, destination, legs) {  
  infoWindows.forEach(w => w.close());  
  infoWindows = [];
```

```
setRouteMapState(true);
```

Fecha e limpa popups de uma busca anterior. Garante que o `<div>` do mapa está visível.

```
directionsService.route(  
  { origin, destination, travelMode: google.maps.TravelMode.TRANSIT },  
  (result, status) => {  
    if (status !== 'OK') return;  
    directionsRenderer.setDirections(result);  
  })
```

`directionsService.route(request, callback)` — pede ao Google Maps JS SDK (lado do cliente, diferente da Routes API v2 do backend) para calcular a rota visual. `travelMode: TRANSIT` = transporte público. No callback, `status !== 'OK'` significa que o Google Maps não conseguiu desenhar essa rota — abandona silenciosamente (a lista de legs/horários já foi mostrada de qualquer forma; só o desenho no mapa fica ausente).

`directionsRenderer.setDirections(result)` desenha a linha da rota no mapa.

```
result.routes[0].legs[0].steps  
  .filter(s => s.travel_mode === 'TRANSIT')  
  .forEach((step, i) => {  
    const leg = legs[i];  
    if (!leg) return;  
    const ptv = leg.ptv;  
    const markerColor = ptv?.found ? '#0052cc' : '#94a3b8';
```

Filtra apenas os "steps" de transporte público (ignora trechos a pé), igual fizemos no backend.

`legs[i]` — usa o **mesmo índice** para casar cada step do Directions (visual) com o `leg` correspondente vindo de `/journey` (que tem os dados da PTV). `if (!leg) return` — proteção caso os arrays tenham tamanhos diferentes (edge case). `markerColor` — azul (`#0052cc`) se temos dados PTV para esse trecho, cinza (`#94a3b8`) se não.

```
const marker = new google.maps.Marker({  
  position: step.start_location,  
  map,  
  icon: {  
    path: google.maps.SymbolPath.CIRCLE,  
    scale: 9,  
    fillColor: markerColor,  
    fillOpacity: 1,  
    strokeColor: 'white',  
    strokeWeight: 2.5
```

```

    },
    title: leg.departure_stop
  });

```

Cria um marcador circular na posição inicial do step ( `step.start_location` — coordenadas calculadas pelo próprio Google Maps Directions), colorido conforme `markerColor`, com tooltip mostrando o nome da parada.

```

let ptvContent = '';
if (ptv?.found && ptv.departures?.length > 0) {
  const next = ptv.departures[0];
  ptvContent = `
    <div style="margin-top:6px;padding-top:6px;border-top:1px solid #f0f0f0;fc
      <strong style="color:#0052cc;">PTV</strong> · Next: <strong>${formatTime
        ${next.platform ? `· Plat ${next.platform}` : ''}
      </div>`;
}

```

Se temos dados PTV com pelo menos uma partida, monta um trecho de HTML extra mostrando "PTV · Next: 09:05 · Plat 3" (a plataforma só aparece `if (next.platform)` — nem toda parada tem plataforma numerada, ex: paradas de tram/ônibus).

```

const infoWindow = new google.maps.InfoWindow({
  content: `
    <div style="font-family:Inter,sans-serif;font-size:13px;max-width:200px;li
      <strong style="color:#1a202c;">${leg.departure_stop}</strong><br/>
      <span style="color:#64748b;">${ICONS[leg.mode] || ' 🚊'} ${leg.line}</sp
      ${ptvContent}
    </div>`
});

marker.addListener('click', () => {
  infoWindows.forEach(w => w.close());
  infoWindow.open(map, marker);
});
infoWindows.push(infoWindow);
});
}
);
}

```

Cria um `InfoWindow` por marcador, mostrando nome da parada, ícone+linha, e o bloco PTV (se houver). Ao clicar, fecha **todos** os outros popups abertos ( `infoWindows.forEach(w => w.close())` ) antes de abrir o atual — evita múltiplos popups sobrepostos. Adiciona ao array de controle.

Nota: aqui o HTML não passa por `esc()` para `leg.departure_stop` / `leg.line` — esses valores vêm do Google Routes API (backend), não de input direto do usuário, então o risco de XSS é baixo, mas idealmente seguiriam o mesmo padrão de `esc()` usado nos POIs.

## Autocomplete de origem/destino

```
let debounceTimers = {};  
  
function setupAutocomplete(inputId) {  
  const input = document.getElementById(inputId);  
  const dropdown = document.getElementById(`${inputId}-dropdown`);  
  let activeIndex = -1;  
  let items = [];
```

`debounceTimers` — objeto que guarda um timer por campo (origem e destino podem ter buscas em andamento independentes). `setupAutocomplete` é chamada uma vez por campo ( `'origin'` e `'destination'` ), criando um closure com seu próprio `activeIndex` (item destacado via teclado) e `items` (sugestões atuais).

```
input.addEventListener('input', () => {  
  clearTimeout(debounceTimers[inputId]);  
  const val = input.value.trim();  
  if (val.length < 2) { dropdown.style.display = 'none'; return; }  
  
  debounceTimers[inputId] = setTimeout(async () => {
```

A cada tecla digitada ( `input` event): cancela qualquer busca pendente ( `clearTimeout` ). Se o texto tem menos de 2 caracteres, esconde o dropdown e não busca (evita resultados sem sentido para "a"). Senão, agenda uma busca em **300ms** — esse é o **debounce**: se o usuário continuar digitando, o timer anterior é cancelado e um novo começa, então a requisição só dispara quando o usuário **para** de digitar por 300ms. Isso evita uma chamada de API a cada tecla.

```
try {  
  const res = await fetch(`/autocomplete?input=${encodeURIComponent(val)}`);  
  const data = await res.json();  
  if (!res.ok) return;
```



```

items = data.suggestions || [];
activeIndex = -1;
if (items.length === 0) { dropdown.style.display = 'none'; return; }
dropdown.innerHTML = items.map((s, i) =>
  `<div class="dropdown-item" data-index="${i}">${s.text}</div>`
).join('');

```

Chama `/autocomplete` (proxy do backend para o Google Places). `items` guarda as sugestões. Reseta `activeIndex` (nenhum item destacado ainda). Se vazio, esconde o dropdown. Senão, monta o HTML: uma `<div>` por sugestão, com `data-index` guardando a posição no array `items` (para recuperar depois).

```

dropdown.querySelectorAll('.dropdown-item').forEach(el => {
  el.addEventListener('mousedown', () => {
    input.value = items[Number(el.dataset.index)].text;
    dropdown.style.display = 'none';
  });
});
dropdown.style.display = 'block';
} catch(e) { console.error(e); }
}, 300);
});

```

Para cada item do dropdown, adiciona listener de `mousedown` (não `click` — `mousedown` dispara **antes** do `blur` do input, evitando que o dropdown já tenha sumido quando o clique seria processado). Ao "clicar", preenche o input com o texto da sugestão e esconde o dropdown. Mostra o dropdown ( `display: block` ). Erros de rede só vão para o console (não interrompem a digitação do usuário).

```

input.addEventListener('keydown', e => {
  const els = dropdown.querySelectorAll('.dropdown-item');
  if (e.key === 'ArrowDown') activeIndex = Math.min(activeIndex + 1, els.length - 1);
  else if (e.key === 'ArrowUp') activeIndex = Math.max(activeIndex - 1, 0);
  else if (e.key === 'Enter') {
    if (activeIndex >= 0 && items[activeIndex]) {
      input.value = items[activeIndex].text;
      dropdown.style.display = 'none';
    } else { search(); }
    return;
  } else if (e.key === 'Escape') { dropdown.style.display = 'none'; }
  els.forEach((el, i) => el.classList.toggle('active', i === activeIndex));
});

```

Navegação por teclado no dropdown:

- `ArrowDown` / `ArrowUp` — incrementa/decrementa `activeIndex`, limitado entre `0` e `els.length - 1` (`Math.min` / `Math.max` evitam sair dos limites do array).
- `Enter` — se há um item destacado (`activeIndex >= 0`), seleciona ele; senão, dispara a busca de rota (`search()`) diretamente — permite o usuário simplesmente digitar e apertar Enter sem usar o mouse.
- `Escape` — fecha o dropdown.
- A última linha aplica a classe CSS `active` apenas ao item correspondente ao `activeIndex` atual (destaque visual), removendo dos demais.

```
input.addEventListener('blur', () =>
  setTimeout(() => { dropdown.style.display = 'none'; }, 150)
);
}

setupAutocomplete('origin');
setupAutocomplete('destination');
```

Ao o input perder o foco (`blur`), esconde o dropdown — mas com **150ms de atraso** via `setTimeout`. Esse atraso é necessário porque, se o usuário clica numa sugestão, o `blur` do input dispara **antes** do `mousedown` do item ser processado; sem o atraso, o dropdown sumiria antes do clique registrar. Chama a função para os dois campos.

## Pontos de Interesse (POIs)

```
const POI_CATEGORIES = [
  'coffee', 'restaurant', 'hotel', 'hospital',
  'public toilet', 'bar', 'night club', 'museum'
];

const POI_COLORS = {
  coffee: '#6f4e37',
  restaurant: '#c62828',
  hotel: '#1565c0',
  hospital: '#2e7d32',
  'public toilet': '#6b7280',
  bar: '#8e24aa',
  'night club': '#ad1457',
  museum: '#ef6c00'
};
```

Lista de categorias disponíveis para o usuário marcar (espelha as chaves de `CATEGORY_MAP` no backend) e a cor associada a cada uma — usada nos marcadores do mapa e na legenda.

```
const selectedPoiPlaces = new Map();
let poiPlaceCache = new Map();
let lastPoiDestination = '';
```

- `selectedPoiPlaces` — `Map` dos lugares que o usuário **selecionou** ("Save place"), chave = `placeKey` (definido mais abaixo), valor = objeto do lugar. Usado tanto para salvar nos favoritos quanto para o contador "Selected places: N".
- `poiPlaceCache` — `Map` com **todos** os lugares retornados pela última busca (selecionados ou não) — permite recuperar os dados completos de um lugar a partir da `placeKey` quando o usuário clica em "Save place".
- `lastPoiDestination` — guarda qual destino foi usado na última busca de POIs (não usado em lógica condicional crítica no trecho mostrado, mas mantido para referência/futura validação).

```
function updatePoiSelectionSummary() {
  const count = selectedPoiPlaces.size;
  document.getElementById('poiSelectionSummary').textContent = `Selected places: ${count}`;
}
```

Atualiza o texto "Selected places: N" sempre que a seleção muda.

```
function getPoiColor(category) {
  return POI_COLORS[String(category || '').toLowerCase()] || '#0052cc';
}
```

Busca a cor da categoria (case-insensitive — `.toLowerCase()`). Se a categoria não existir no mapa (`undefined`), usa azul padrão `#0052cc`.

```
function renderPoiLegend(data) {
  const legendEl = document.getElementById('poiLegend');
  if (!legendEl) return;

  const categories = (data.categories || []).
    .map(category => category.category || category.label)
    .filter(Boolean);
  const uniqueCategories = [...new Set(categories)];

  legendEl.innerHTML = uniqueCategories.map(category => `
    <span class="poi-legend-item">
```

```

    <span class="poi-legend-swatch" style="background:${getPoiColor(category)}"></span>
    ${esc(category)}
  </span>
  `).join('');
}

```

Constrói a legenda de cores (uma "bolinha" colorida + nome da categoria, para cada categoria que retornou resultados). `category.category || category.label` — usa o identificador interno se disponível, senão o rótulo legível. `new Set(...)` remove duplicatas (embora cada categoria já seja única na resposta, é uma proteção extra). `esc(category)` escapa o texto antes de inserir no HTML.

```

function togglePoiPlace(placeKey) {
  const place = poiPlaceCache.get(placeKey);
  if (!place) return;
  if (selectedPoiPlaces.has(placeKey)) {
    selectedPoiPlaces.delete(placeKey);
  } else {
    selectedPoiPlaces.set(placeKey, place);
  }
  updatePoiSelectionSummary();
}

```

Alterna a seleção de um lugar: busca o objeto completo no cache; se já estava selecionado, remove (desmarcar); senão, adiciona (marcar). Atualiza o contador.

```

const btn = document.querySelector(`[data-place-key="${placeKey.replace(/"/g, '"');"}]`);
if (btn) {
  const selected = selectedPoiPlaces.has(placeKey);
  btn.classList.toggle('selected', selected);
  btn.textContent = selected ? 'Selected' : 'Save place';
}
autoSavePlacesToFav();
}

```

Encontra o botão correspondente via seletor de atributo `[data-place-key="..."]`. `placeKey.replace(/"/g, '"')` escapa aspas duplas dentro do `placeKey` para não quebrar o seletor CSS (caso o `place_id` ou nome contenha `"`). Atualiza visualmente o botão (classe `selected` + texto). Chama `autoSavePlacesToFav()` — se o usuário chegou aqui a partir de uma rota salva ( `favId` na URL), salva automaticamente a seleção atualizada.

```

document.getElementById('poiResults').addEventListener('click', event => {
  const btn = event.target.closest('.poi-select-btn');
}

```

```

    if (!btn) return;
    const placeKey = btn.dataset.placeKey;
    if (!placeKey) return;
    togglePoiPlace(placeKey);
  });

```

**Event delegation:** em vez de adicionar um listener de clique a cada botão "Save place" individualmente (que seriam recriados a cada busca), adiciona **um único listener** no contêiner pai ( `#poiResults` ) que captura cliques em qualquer descendente. `event.target.closest('.poi-select-btn')` — encontra o elemento `.poi-select-btn` mais próximo a partir de onde o clique realmente aconteceu (cobre o caso de clicar num `<span>` dentro do botão). Se o clique não foi em um botão de seleção, ignora.

```

function getSelectedPoiPlaces() {
  return Array.from(selectedPoiPlaces.values());
}

function getSelectedCategories() {
  return Array.from(document.querySelectorAll('#poiCategories input:checked'))
    .map(input => input.value)
    .filter(Boolean);
}

```

- `getSelectedPoiPlaces()` — converte os `values()` do Map `selectedPoiPlaces` em array (formato esperado pelo backend ao salvar favoritos).
- `getSelectedCategories()` — lê todos os checkboxes marcados dentro de `#poiCategories`, extrai seus `value`s, remove valores vazios/falsy.

```

function renderPoiPlaceItem(place) {
  const placeKey = place.place_id || `${place.name}:${place.address}`;
  const selected = selectedPoiPlaces.has(placeKey);
  poiPlaceCache.set(placeKey, place);
}

```

`placeKey` — identificador único do lugar: usa `place_id` do Google se disponível, senão monta uma chave a partir de nome+endereço (fallback para lugares sem `place_id`). Verifica se já está selecionado (em re-renderizações). Salva o objeto completo no cache para uso posterior em `togglePoiPlace`.

```

const openText = place.open_now === true ? 'Open now' : place.open_now === false ? 'Closed' : 'Open now';
const ratingText = place.rating ? `★ ${place.rating} (${place.user_ratings_total} reviews)` : 'Rating';
const distanceText = place.distance_meters != null ? `${place.distance_meters} m away` : 'Distance';

```

```
const walkText = place.walking_minutes != null ? `${place.walking_minutes} min walk` : ''
const extraText = [distanceText, walkText].filter(Boolean).join(' · ');
```

Prepara textos derivados:

- `openText` — três estados possíveis ( `true` / `false` / `null` — comparação estrita `===` distingue os três).
- `ratingText` — só mostra se há rating; `user_ratings_total || 0` evita "undefined" na contagem.
- `distanceText` / `walkText` — só mostram se os valores existem ( `!= null` ).
- `extraText` — junta distância e tempo de caminhada com `" · "`, mas `.filter(Boolean)` remove qualquer um que esteja vazio (evita `" · 5 min walk"` com separador sobrando se `distanceText` for `''`).

```
return `
  <div class="leg-card" style="margin-bottom:10px;">
    <div class="leg-header">
      <div class="leg-step">•</div>
      <div class="leg-info">
        <div class="leg-line">${place.name}</div>
        <div class="leg-stops">${place.address || 'Address unavailable'}</div>
      </div>
      <div class="leg-mode" style="min-width:110px;">
        <span class="mode-chip" style="background:#ffcd00;color:#001a52;">${openText}</span>
      </div>
    </div>
    <div class="leg-body" style="padding-top:6px;">
      ${extraText ? `<div style="margin-top:10px;font-size:13px;color:#0f172a;font-weight:bold;">${extraText}</div>`}
      ${ratingText ? `<div style="margin-top:6px;font-size:13px;color:#475569;">${ratingText}</div>`}
      <button type="button" class="poi-select-btn${selected ? ' selected' : ''}" data-place-key=${place.key}
        ${selected ? 'Selected' : 'Save place'}
      </button>
    </div>
  </div>`;
}
```

Monta o card HTML de um lugar, reutilizando as classes visuais `.leg-card` / `.leg-header` / `.leg-mode` (mesmo estilo dos cards de trechos de viagem, para consistência visual). Inclui `extraText` / `ratingText` condicionalmente (só se não vazios). O botão recebe `data-place-key` (lido pelo event delegation acima) e classe `selected` + texto conforme estado atual.

```
function renderPoiResults(data) {
  const container = document.getElementById('poiResults');
```

```

if (!data.categories || data.categories.length === 0) {
  container.innerHTML = '<div class="empty-state">No POI categories returned.</div>';
  return;
}

container.innerHTML = data.categories.map(category => {
  const places = category.places || [];
  return `
    <div class="section-head" style="margin-top:20px;">
      <h2>${category.label}</h2>
      <span class="section-badge">${places.length} result${places.length !== 1 ? 's' : ''}</span>
    </div>
    ${places.length === 0 ? '<div class="empty-state">No places found for this category.</div>' : ''}
  `;
}).join('');
}

```

Renderiza todas as categorias retornadas. Para cada uma: cabeçalho com o nome da categoria e contagem de resultados ( `"5 results"` ou `"1 result"` — pluralização condicional `places.length !== 1 ? 's' : ''` ). Se não há lugares naquela categoria, mostra mensagem vazia; senão, renderiza cada lugar via `renderPoiPlaceItem` .

```

async function searchTargetPlaces() {
  const destination = document.getElementById('destination').value.trim();
  const statusEl    = document.getElementById('poiStatus');
  const resultsEl   = document.getElementById('poiResults');
  const btn         = document.getElementById('poiSearchBtn');
  const categories  = getSelectedCategories();

  if (!destination) {
    document.getElementById('poiSection').style.display = 'block';
    statusEl.innerHTML = '<span style="color:#dc2626;font-size:12px;">Please enter the destination';
    return;
  }

  if (categories.length === 0) {
    document.getElementById('poiSection').style.display = 'block';
    statusEl.innerHTML = '<span style="color:#dc2626;font-size:12px;">Select at least one category';
    return;
  }
}

```

Função chamada pelo botão "Find places". Lê o destino atual e as categorias marcadas. Validações: precisa de um destino preenchido e pelo menos uma categoria selecionada — em ambos os casos, **mostra a seção de POIs** ( `poiSection.style.display = 'block'` ) mesmo no erro, para que a mensagem vermelha seja visível.

```

document.getElementById('poiSection').style.display = 'block';
btn.disabled = true;
btn.textContent = 'Searching...';
statusEl.innerHTML = `<span style="color:var(--text-muted);font-size:12px;">Searching
resultsEl.innerHTML = '';

try {
  const res = await fetch(`/target-places?destination=${encodeURIComponent(destination)}`);
  const data = await res.json();
  if (!res.ok) {
    const msg = typeof data.error === 'object' ? JSON.stringify(data.error, null, 2) : data.error;
    statusEl.innerHTML = `<span style="color:#dc2626;font-size:12px;">${msg}</span>`;
    return;
  }
}

```

Mostra estado de "carregando" (botão desabilitado, texto "Searching...", limpa resultados antigos). Chama `/target-places` com `destination` e `categories` (lista separada por vírgula) na query string. Se erro: `data.error` pode ser uma string ou um objeto (ex: erro retornado pelo Axios do backend); `typeof data.error === 'object'` decide se precisa de `JSON.stringify` para exibir.

```

statusEl.innerHTML = `<span style="color:#166534;font-size:12px;font-weight:600;">Feito!</span>`;
selectedPoiPlaces.clear();
renderPoiResults(data);
renderPoiLegend(data);
updatePoiSelectionSummary();
lastPoiDestination = destination;
await loadMapsApi();
showPoiOnExistingMap(data);

```

- `data.categories.reduce((sum, c) => sum + (c.places?.length || 0), 0)` — soma o total de lugares encontrados em todas as categorias ( `reduce` acumula; `c.places?.length || 0` trata categorias sem `places` ).
- `selectedPoiPlaces.clear()` — uma nova busca **reseta** as seleções anteriores (a busca anterior pode ter sido para um destino diferente).
- Renderiza a lista, a legenda, atualiza o contador, guarda o destino usado.
- `await loadMapsApi()` — garante que o mapa está carregado antes de tentar colocar marcadores.
- `showPoiOnExistingMap(data)` — desenha os marcadores de POI no mapa.

```

} catch (err) {
  statusEl.innerHTML = `<span style="color:#dc2626;font-size:12px;">Request failed: ${err.message}</span>`;
}

```



```

    } finally {
      btn.disabled = false;
      btn.textContent = 'Find places';
    }
  }
}

```

Erro de rede (ex: servidor offline) — mostra mensagem com `err.message`. `finally` sempre restaura o botão, mesmo em caso de erro.

## Renderizar um trecho da viagem

```

function renderLeg(leg, index, total) {
  const icon      = ICONS[leg.mode]      || '🚆';
  const modeLabel = MODE_LABEL[leg.mode] || leg.mode;
  const modeColor = MODE_COLOR[leg.mode] || '#555';
  const ptv       = leg.ptv;

```

`leg` é um trecho retornado por `/journey` (com `ptv` e `disruptions` já anexados pelo backend). `index` / `total` — posição deste trecho e total de trechos (usado para saber se é o último). Busca ícone/rótulo/cor com fallbacks ( `|| '🚆'`, `|| leg.mode`, `|| '#555'` ) caso o `mode` não esteja nas tabelas.

```

let sourceTag = '';
let timesHtml = '';

if (ptv?.found && ptv.departures?.length > 0) {
  sourceTag = '<span class="source-badge ptv">PTV real-time</span>';
  timesHtml = `<div class="departure-list">${ptv.departures.map(d => {
    const num      = d.route_number || d.route_name || '?';
    const minsUntil = d.mins_until;
    let minsClass = '';
    let minsText  = '';
    if (minsUntil <= 0) { minsClass = 'now'; minsText = 'Now'; }
    else if (minsUntil <= 3) { minsClass = 'soon'; minsText = `${minsUntil} min`; }
    else { minsText = `${minsUntil} min`; }
    return `
    <div class="departure-row">
      <span class="route-chip">${num}</span>
      <span class="dep-time">${formatTime(d.scheduled_departure)}</span>
      ${d.platform ? `<span class="dep-platform">Plat ${d.platform}</span>` : ''}
      <span class="dep-mins ${minsClass}">${minsText}</span>

```

```

    </div>`;
  }).join('')}</div>`;

```

Se a PTV encontrou a parada e tem partidas: mostra a etiqueta "PTV real-time" e renderiza a lista de próximas partidas. Para cada partida `d`:

- `num` — número/nome da rota, ou `'?'` se ambos ausentes.
- Classificação visual por urgência: `minsUntil <= 0` → texto "Now" com classe `now` (provavelmente destaque vermelho/pulsante); `<= 3` → classe `soon` (atenção); senão, texto normal `"N min"`.
- Cada linha mostra: chip da rota, horário formatado, plataforma (se houver), e o texto de "quanto falta" com a classe de urgência.

```

} else if (ptv?.found) {
  sourceTag = '<span class="source-badge ptv">PTV real-time</span>';
  timesHtml = `<p style="color:var(--text-muted);font-size:13px;">No direct services f
} else {
  timesHtml = `<p style="color:var(--text-light);font-size:13px;font-style:italic;">PT
}

```

- `ptv?.found` mas sem `departures` — a parada existe na PTV mas não há partidas futuras retornadas (ex: fim do serviço). Ainda mostra a etiqueta PTV, mas com mensagem explicativa.
- `!ptv?.found` — a busca por nome falhou ( `searchStop` não achou a parada). Sem etiqueta PTV, mensagem em itálico avisando que os horários não estão disponíveis.

```

const transferNote = (total > 1 && index < total - 1)
  ? `<p class="transfer-note">Transfer at <strong>${leg.arrival_stop}</strong></p>`
  : '';

```

Se há mais de um trecho ( `total > 1` ) e este **não é o último** ( `index < total - 1` ), mostra "Transfer at [parada de chegada]" — informa ao usuário onde fazer a baldeação para o próximo trecho.

```

const disruptions = leg.disruptions || [];
const disruptionsHtml = disruptions.length > 0
  ? disruptions.map(d => {
    const isDelay = d.type === 'delay' ||
      /delay|late|minutes?\s+late/i.test((d.title || '') + ' ' + (d.description || ''));
    return `
    <div class="alert-bar ${isDelay ? 'alert-delay' : 'alert-disruption'}">
      <span class="alert-icon">${isDelay ? '🕒' : '⚠️'}</span>
    </div>

```

```

        <strong>${d.title}</strong>
        ${d.description ? `<p>${d.description}</p>` : ''}
    </div>
</div>`;
}).join('')
: ptv?.found && ptv.departures?.length > 0
? `<p style="color:#16a34a;font-size:12px;font-weight:500;margin-top:6px;">Running
: '';

```

- `leg.disruptions` vem de `getAlertsFromDeparture` no backend — array com 0 ou 1 item ( `type: 'delay'` ).
- `isDelay` — checa `d.type === 'delay'` (já setado pelo backend) **ou** uma regex extra como segunda checagem ( `/delay|late|minutes?\s+late/i` — cobre "minute late"/"minutes late"). Essa redundância serve como rede de segurança caso `disruptions` venha de outra fonte no futuro (ex: `commonAlerts` ) sem o campo `type` corretamente setado.
- Se há disruptions: renderiza uma `.alert-bar` por item, com classe `alert-delay` (ícone 🕒) ou `alert-disruption` (ícone ⚠️) conforme `isDelay` .
- Se **não** há disruptions **e** a PTV retornou partidas ( `ptv?.found && ptv.departures?.length > 0` ) — mostra "Running on time" em verde. Se não há dados PTV de jeito nenhum, não mostra nada (string vazia) — não faria sentido dizer "no horário" sem dados de comparação.

```

return `
<div class="leg-card">
  <div class="leg-header">
    <div class="leg-step">${index + 1}</div>
    <div class="leg-info">
      <div class="leg-line">${leg.line} – ${leg.headsign}</div>
      <div class="leg-stops">${leg.departure_stop}<span>→</span>${leg.arrival_stop}</div>
    </div>
    <div class="leg-mode">
      <div class="leg-mode-icon">${icon}</div>
      <span class="mode-chip" style="background:${modeColor}">${modeLabel}</span>
    </div>
  </div>
  <div class="leg-body">
    ${sourceTag}
    ${timesHtml}
    ${transferNote}
    ${disruptionsHtml}
  </div>
</div>`;
}

```

Monta o card final do trecho: número do passo ( `index + 1` , já que arrays começam em 0), nome da linha + destino (headsign), paradas de embarque/desembarque com seta  , ícone+chip do modo de transporte, e o corpo com etiqueta de fonte, horários, nota de transferência e alertas/atrasos.

## Busca principal de rota

```
async function search() {
  const origin      = document.getElementById('origin').value.trim();
  const destination = document.getElementById('destination').value.trim();
  const btn         = document.getElementById('searchBtn');
  const statusEl    = document.getElementById('status');
  const resultsEl   = document.getElementById('results');

  if (!origin || !destination) {
    statusEl.innerHTML = '<span class="status-error">Please enter both origin and destination</span>';
    return;
  }
}
```

Lê os campos, valida que ambos foram preenchidos.

```
btn.disabled = true;
btn.textContent = 'Searching...';
statusEl.innerHTML = `<span class="status-loading"><span class="spinner"></span>Finding route</span>`;
resultsEl.style.display = 'none';

try {
  const res = await fetch(`/journey?origin=${encodeURIComponent(origin)}&destination=${encodeURIComponent(destination)}`);
  const data = await res.json();

  if (!res.ok) {
    const msg = typeof data.error === 'object' ? JSON.stringify(data.error, null, 2) : data.error;
    statusEl.innerHTML = `<span class="status-error">${msg}</span>`;
    return;
  }
}
```

Estado de loading (spinner visível, resultados anteriores escondidos). Chama `/journey` . Erro: mesma lógica de tratar `data.error` como string ou objeto.

```
statusEl.innerHTML = '';
document.getElementById('totalDuration').textContent = formatDuration(data.duration_
```

```

document.getElementById('totalDistance').textContent = formatDistance(data.distance_
document.getElementById('legCount').textContent      = `${data.legs.length} leg${dat

const legsEl = document.getElementById('legs');
legsEl.innerHTML = data.legs.length === 0
  ? '<div class="empty-state">No transit legs found for this route.</div>'
  : data.legs.map((leg, i) => renderLeg(leg, i, data.legs.length)).join('');

resultsEl.style.display = 'block';

```

Limpa a mensagem de status. Preenche o resumo (duração total, distância total, contagem de trechos com pluralização). Renderiza cada trecho via `renderLeg`, ou mostra mensagem vazia se não há trechos de transporte (ex: rota inteira a pé). Mostra a seção de resultados.

```

// Show save banner
_lastOrigin = origin;
_lastDestination = destination;
const banner = document.getElementById('saveBanner');
const successEl = document.getElementById('saveSuccess');
const saveBtn = document.getElementById('saveBannerBtn');
document.getElementById('saveNameInput').value = '';
successEl.style.display = 'none';
saveBtn.style.display = '';
if (_token) {
  banner.style.display = 'flex';
} else {
  banner.style.display = 'flex';
  document.getElementById('saveNameInput').placeholder = 'Login to save routes';
  document.getElementById('saveNameInput').disabled = true;
  saveBtn.textContent = 'Login';
}

```

Salva `origin` / `destination` em variáveis globais (usadas por `saveJourney()`). Reseta o estado do banner de "salvar rota": limpa o campo de nome, esconde mensagem de sucesso anterior, mostra o botão. Se logado, mostra o banner normal. Se **não** logado, mostra o banner mas com o campo desabilitado, placeholder "Login to save routes" e o botão com texto "Login" — incentiva o cadastro sem esconder a funcionalidade.

```

const hasPtvData = data.legs.some(l => l.ptv?.found);
if (hasPtvData) {
  showRouteOnMap(origin, destination, data.legs);
} else {
  setRouteMapState(false);
}

```

```

    }

    } catch (err) {
      statusEl.innerHTML = `<span class="status-error">Request failed: ${err.message}</span>`;
    } finally {
      btn.disabled = false;
      btn.textContent = 'Search →';
    }
  }
}

```

`data.legs.some(1 => 1.ptv?.found)` — verifica se **pelo menos um** trecho tem dados PTV válidos. Se sim, desenha o mapa com a rota ( `showRouteOnMap` ). Se não (nenhum trecho tem PTV — situação rara, mas possível), mostra o estado "vazio" do mapa ( `setRouteMapState(false)` ) em vez de um mapa sem informação útil. Erro de rede mostrado ao usuário; `finally` sempre restaura o botão de busca.

## Pré-preenchimento via URL e auto-save

```

// Pre-fill from URL params (e.g. coming from favorites "Plan" button)
const _urlParams = new URLSearchParams(window.location.search);
const _favId = _urlParams.get('favId') || null;

```

`URLSearchParams(window.location.search)` faz parse da query string da URL atual (ex: `?origin=X&destination=Y&favId=abc123` ). `_favId` — se presente, indica que o usuário chegou aqui pelo botão "Plan" de uma rota salva (em `favorites.html` ); usado depois para auto-salvar POIs nessa rota específica.

```

(function prefillFromParams() {
  const o = _urlParams.get('origin');
  const d = _urlParams.get('destination');
  if (o) document.getElementById('origin').value = o;
  if (d) document.getElementById('destination').value = d;
  if (o && d) search();
})();

```

**IIFE** (Immediately Invoked Function Expression) — função que roda imediatamente ao ser definida, usada aqui para não poluir o escopo global com variáveis temporárias ( `o` , `d` ). Se a URL trouxe `origin` / `destination` , preenche os campos. Se **ambos** vieram, dispara `search()` automaticamente — é assim que clicar em "Plan" numa rota salva já mostra os resultados sem o usuário precisar clicar em "Search" de novo.

```
let _autoSaveTimer = null;
async function autoSavePlacesToFav() {
  if (!_favId || !_token) return;
  clearTimeout(_autoSaveTimer);
  _autoSaveTimer = setTimeout(async () => {
```

Se não há `_favId` (não veio de uma rota salva) ou não está logado, não faz nada. Mesmo padrão de **debounce** do autocomplete: cancela o timer anterior e agenda um novo — se o usuário marcar/desmarcar vários POIs rapidamente, só salva uma vez, 600ms depois da última mudança.

```
try {
  await fetch(`/favorites/${_favId}`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${_token}` },
    body: JSON.stringify({ places: getSelectedPoiPlaces() })
  });
  const indicator = document.getElementById('favAutoSaveIndicator');
  if (indicator) {
    indicator.textContent = 'Places saved to route';
    indicator.style.opacity = '1';
    setTimeout(() => { indicator.style.opacity = '0'; }, 2000);
  }
} catch { /* silent */ }
}, 600);
}
```

`PATCH /favorites/:id` enviando **só** o campo `places` (atualização parcial — o backend mantém `name` / `origin` / `destination` inalterados, conforme vimos em `routes/favorites.js`). Sucesso: mostra um indicador "Places saved to route" por 2 segundos (fade out via `opacity`). Erro: ignorado silenciosamente ( `/* silent */` ) — é uma ação em segundo plano, não vale interromper o fluxo do usuário com um alerta por uma falha de auto-save.

```
loadMapsApi();
```

Última linha do script — inicia o carregamento do Google Maps assim que a página carrega, **mesmo antes** de qualquer busca, para que o mapa esteja pronto quando o usuário pesquisar.

---

## 16. public/register.html — `<script>` (Cadastro)

---

```
// Redirect if already logged in
if (localStorage.getItem('ptv_token')) {
  window.location.href = '/favorites.html';
}
```

Roda assim que o script carrega (não é uma função). Se já existe um token salvo (usuário já logado), redireciona imediatamente para `favorites.html` — não faz sentido mostrar a tela de cadastro para quem já tem conta logada.

```
async function handleRegister(e) {
  e.preventDefault();
```

`e` é o evento de submit do formulário. `e.preventDefault()` impede o comportamento padrão do HTML (recarregar a página enviando o form via GET/POST tradicional) — queremos controlar o envio via `fetch`.

```
const btn = document.getElementById('submitBtn');
const errorEl = document.getElementById('errorMsg');
const email = document.getElementById('email').value.trim();
const password = document.getElementById('password').value;
const confirm = document.getElementById('confirm').value;

errorEl.style.display = 'none';
```

Lê os três campos do formulário (email, senha, confirmação de senha). Esconde qualquer mensagem de erro de uma tentativa anterior.

```
if (password !== confirm) {
  errorEl.textContent = 'Passwords do not match';
  errorEl.style.display = 'block';
  return;
}
```

Validação **no frontend**: se a senha e a confirmação não baterem, mostra erro e cancela — evita uma chamada de API desnecessária para um erro que pode ser detectado localmente. (O backend não recebe/valida `confirm` — é só uma conveniência de UX.)

```
btn.disabled = true;
btn.textContent = 'Creating account...';

try {
```



```
const res = await fetch('/auth/register', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password })
});
const data = await res.json();
```

Estado de loading no botão. `POST /auth/register` enviando `email` e `password` (note que `confirm` **não** é enviado — só serviu para a validação local).

```
if (!res.ok) {
  errorEl.textContent = data.error;
  errorEl.style.display = 'block';
  return;
}
```

Se o backend retornou erro (ex: **409** "Email already registered", **400** validação), mostra a mensagem vinda do backend ( `data.error` ).

```
localStorage.setItem('ptv_token', data.token);
localStorage.setItem('ptv_user', JSON.stringify(data.user));
window.location.href = '/favorites.html';
} catch (err) {
  errorEl.textContent = 'Request failed. Please try again.';
  errorEl.style.display = 'block';
} finally {
  btn.disabled = false;
  btn.textContent = 'Create account';
}
}
```

Sucesso: salva o token JWT e os dados do usuário no `localStorage` ( `JSON.stringify` porque `localStorage` só armazena strings) e redireciona para `favorites.html` — o usuário já está logado automaticamente, sem precisar passar pela tela de login. Erro de rede: mensagem genérica. `finally` : sempre restaura o botão (mesmo que o redirecionamento abaixo geralmente "ganhe" antes — mas se o `fetch` falhar, o botão precisa voltar ao normal).

---